



UNIVERSITY OF THESSALY  
SCHOOL OF ENGINEERING  
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

## **Building an AI Chatbot using RAG Architecture**

Diploma Thesis

**Alexandros Sialtsis**

**Supervisor:** Hariklia Tsalapata

February 2026





UNIVERSITY OF THESSALY  
SCHOOL OF ENGINEERING  
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

## **Building an AI Chatbot using RAG Architecture**

Diploma Thesis

**Alexandros Sialtsis**

**Supervisor:** Hariklia Tsalapata

February 2026





ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ

ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ

ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

## **Ανάπτυξη AI Chatbox με RAG Αρχιτεκτονική**

Διπλωματική Εργασία

**Αλέξανδρος Σιάλτσης**

**Επιβλέπουσα:** Χαρίκλεια Τσαλαπάτα

Φεβρουάριος 2026



Approved by the Examination Committee:

Supervisor **Hariklia Tsalapata**

Professor, Department of Electrical and Computer Engineering, University of Thessaly

Member **Georgios Thanos**

Professor, Department of Electrical and Computer Engineering, University of Thessaly

Member **Georgios Stamoulis**

Professor, Department of Electrical and Computer Engineering, University of Thessaly





# Acknowledgements

I would like to express my deepest gratitude to my thesis supervisor, Harikleia Tsalapata, for her invaluable guidance, support, and patience throughout the completion of this thesis. Her insight and encouragement were instrumental at every stage of this work.

I would also like to sincerely thank all my professors at the University of Thessaly, who passed on their knowledge and helped shape both my academic path and my way of thinking, ultimately bringing me to where I am today.

From the bottom of my heart, I thank my mother and my father for their selfless emotional and financial support. Their unwavering belief in me, their sacrifices, and the values they instilled have shaped me into the person I am today.

I am grateful to all the friends I made during my years as a student in Volos. I deeply cherish the memories, the experiences, and all the moments we shared together. These years will remain with me, remembered fondly and all too well.

As this important chapter of my life comes to a close, many new ones are waiting to begin. I move forward with gratitude for the past and excitement for what lies ahead.



## **DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS**

«Being fully aware of the implications of copyright laws, I expressly state that this diploma thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I also declare that the results of the work have not been used to obtain another degree. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism».

The declarant

Alexandros Sialtsis

# Diploma Thesis

## Building an AI Chatbot using RAG Architecture

Alexandros Sialtsis

### Abstract

University websites have a lot of different and often changing information, which makes it hard for students to find the right answers using regular search tools. Large Language Models (LLMs) enable natural language interaction, they are prone hallucination when they don't have access to accurate domain knowledge. Retrieval-Augmented Generation (RAG) solves this problem by using outside information sources to support the responses it creates.

This thesis presents the design and evaluation of a RAG-based chatbot developed for e-ce.uth.gr. The system is built using scraped content from the website and incorporates semantic HTML-based chunking, dense vector embeddings, metadata-enriched indexing, and a hybrid retrieval strategy that combines vector similarity with keyword-based search. Multiple retrieval configurations are evaluated, including a baseline size-based chunking approach and an enhanced pipeline employing semantic chunking, metadata-aware retrieval, and re-ranking.

Experimental findings show that semantic chunking and hybrid retrieval significantly enhance retrieval recall and answer relevance in comparison to the baseline system. The findings underscore the significance of retrieval design decisions in RAG architectures and offer pragmatic guidance for implementing dependable, domain-specific chatbots in higher education settings.

### Keywords:

Retrieval-Augmented Generation, Large Language Model, Dense Vector Embeddings, Semantic Search, Hybrid Information Retrieval

## Διπλωματική Εργασία

### Ανάπτυξη AI Chatbox με RAG Αρχιτεκτονική

Αλέξανδρος Σιάλτσης

## Περίληψη

Τα πανεπιστήμια έχουν πολλές διαφορετικές και μεταβαλλόμενες πληροφορίες στις ιστοσελίδες τους, κάνοντας το δύσκολο για φοιτητές να βρουν τις απαντήσεις στις ερωτήσεις τους, χρησιμοποιώντας τους κλασσικούς τρόπους αναζήτησης. Αν και, τα Large Language Models (LLMs) κάνουν δυνατή την επικοινωνία με φυσική γλώσσα, είναι επιρρεπή σε hallucinations, όταν δεν έχουν πρόσβαση στην ακριβή πληροφορία. Η Retrieval-Augmented Generation (RAG) αρχιτεκτονική, λύνει αυτό το πρόβλημα, τροφοδοτώντας το LLM την απαιτούμενη πληροφορία για να στηρίξει την απάντηση που παράγει.

Αυτή η διπλωματική παρουσιάζει την αρχιτεκτονική ενός RAG-based chatbot για το e-ce.uth.gr. Το σύστημα φτιάχτηκε χρησιμοποιώντας υλικό από την ιστοσελίδα, ενσωματώνοντας σημασιολογικό διαχωρισμό του HTML, embedding πυκνών διανυσμάτων, εμπλουτισμένο με metadata indexing και μία υβριδική στρατηγική ανάκτησης πληροφορίας που συνδιάζει ομοιότητα διανυσμάτων και αναζήτηση με λέξεις-κλειδιά. Πολλές αρχιτεκτονικές δοκιμάστηκαν, συμπεριλαμβανομένου μίας αρχικής με διαχωρισμό σε κομμάτια ίδιου μεγέθους και μίας βελτιωμένης εκδοχής που χρησιμοποιεί σημασιολογικό διαχωρισμό, αναζήτηση με metadata και αναθμολόγησης.

Τα αποτελέσματα δείχνουν ότι ο σημασιολογικός διαχωρισμός και η υβριδική ανάκτηση βελτιώνουν σημαντικά την ανάκληση της πληροφορίας και την σχετικότητα των απαντήσεων συγκριτικά με το βασικό σύστημα. Τα ευρήματα αυτά αναδεικνύουν τη σημασία των αρχιτεκτονικών αποφάσεων στην ανάκτηση πληροφορίας σε Retrieval-Augmented Generation (RAG) αρχιτεκτονικές και προσφέρουν καθοδήγηση για την υλοποίηση αξιόπιστων και εξειδικευμένων chatbot σε ιστοσελίδες.



# Table of contents

|                                                    |              |
|----------------------------------------------------|--------------|
| <b>Acknowledgements</b>                            | <b>ix</b>    |
| <b>Abstract</b>                                    | <b>xii</b>   |
| <b>Περίληψη</b>                                    | <b>xiii</b>  |
| <b>Table of contents</b>                           | <b>xv</b>    |
| <b>List of figures</b>                             | <b>xix</b>   |
| <b>List of tables</b>                              | <b>xxi</b>   |
| <b>Abbreviations</b>                               | <b>xxiii</b> |
| <b>1 Introduction</b>                              | <b>1</b>     |
| 1.1 Paper Subject . . . . .                        | 1            |
| 1.1.1 Contribution . . . . .                       | 2            |
| 1.2 Organization . . . . .                         | 3            |
| <b>2 Literature Review</b>                         | <b>5</b>     |
| 2.1 Chatbots in Higher Education . . . . .         | 5            |
| 2.2 Large Language Models (LLMs) . . . . .         | 6            |
| 2.3 Information Retrieval . . . . .                | 7            |
| 2.4 Retrieval-Augmented Generation (RAG) . . . . . | 8            |
| <b>3 Implementation</b>                            | <b>11</b>    |
| 3.1 Data Collection . . . . .                      | 13           |
| 3.1.1 Website Scraping Process . . . . .           | 13           |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| 3.1.2    | Semantic Chunking Strategy . . . . .                    | 14        |
| 3.2      | Embedding and Indexing . . . . .                        | 15        |
| 3.2.1    | Embedding and Vector Representation . . . . .           | 15        |
| 3.2.2    | Vector Indexing Using FAISS . . . . .                   | 17        |
| 3.2.3    | Lexical Retrieval Using BM25 . . . . .                  | 18        |
| 3.3      | Hybrid Query Logic . . . . .                            | 19        |
| 3.3.1    | Dense Vector Retrieval . . . . .                        | 19        |
| 3.3.2    | Lexical BM25 Retrieval . . . . .                        | 20        |
| 3.3.3    | Candidate Set Fusion . . . . .                          | 20        |
| 3.4      | Reranking . . . . .                                     | 21        |
| 3.4.1    | LLM-Based Relevance Scoring Using GPT-4o-mini . . . . . | 21        |
| 3.4.2    | Prompt Engineering . . . . .                            | 21        |
| 3.4.3    | Prompt Engineering Considerations . . . . .             | 22        |
| 3.4.4    | Integration into the Retrieval Pipeline . . . . .       | 22        |
| 3.5      | Answer Generation . . . . .                             | 23        |
| 3.5.1    | Context Construction . . . . .                          | 23        |
| 3.5.2    | Answer Generation Prompt . . . . .                      | 23        |
| 3.5.3    | Prompt Engineering . . . . .                            | 24        |
| 3.5.4    | Model Configuration . . . . .                           | 25        |
| <b>4</b> | <b>Comparative Analysis of Retrieval Architectures</b>  | <b>27</b> |
| 4.1      | One Index - Predefined Chunking . . . . .               | 27        |
| 4.1.1    | Content Extraction and Preprocessing . . . . .          | 28        |
| 4.1.2    | Fixed-Size Token-Based Chunking . . . . .               | 28        |
| 4.1.3    | Embedding and Index Construction . . . . .              | 29        |
| 4.1.4    | Query Processing and Retrieval . . . . .                | 29        |
| 4.1.5    | Characteristics and Limitations . . . . .               | 29        |
| 4.2      | Category-Based Multi-Index Retrieval Strategy . . . . . | 30        |
| 4.2.1    | Design Motivation . . . . .                             | 30        |
| 4.2.2    | Data Collection via Sitemap Parsing . . . . .           | 30        |
| 4.2.3    | Rule-Based URL Categorization . . . . .                 | 31        |
| 4.2.4    | Chunking, Embedding, and Index Construction . . . . .   | 31        |
| 4.2.5    | Query Classification and Routing . . . . .              | 32        |



|          |                                                                      |           |
|----------|----------------------------------------------------------------------|-----------|
| 4.2.6    | Category-Specific Retrieval and Answer Generation . . . . .          | 32        |
| 4.2.7    | Analysis of the Multi-Index Retrieval Strategy Performance . . . . . | 32        |
| 4.3      | Hybrid Metadata-Based Retrieval Strategy . . . . .                   | 34        |
| 4.3.1    | Unified Index with Semantic Density Preservation . . . . .           | 34        |
| 4.3.2    | Metadata-Enriched Chunk Representation . . . . .                     | 35        |
| 4.3.3    | Support for Multi-Faceted Queries . . . . .                          | 36        |
| 4.3.4    | Robustness and Design Advantages . . . . .                           | 36        |
| <b>5</b> | <b>Evaluation</b>                                                    | <b>37</b> |
| 5.1      | Evaluation Dataset . . . . .                                         | 37        |
| 5.2      | Evaluation Metrics . . . . .                                         | 38        |
| 5.2.1    | Retrieval Metrics . . . . .                                          | 39        |
| 5.2.2    | Answer Quality Metrics . . . . .                                     | 40        |
| 5.3      | Evaluation Results . . . . .                                         | 40        |
| 5.3.1    | Retrieval Performance . . . . .                                      | 40        |
| 5.3.2    | Answer Quality Evaluation . . . . .                                  | 41        |
| 5.4      | Error Analysis . . . . .                                             | 42        |
| 5.4.1    | Retrieval Failures . . . . .                                         | 43        |
| 5.4.2    | Answer Generation Errors . . . . .                                   | 44        |
| 5.4.3    | Summary of Failure Modes . . . . .                                   | 45        |
| <b>6</b> | <b>User Interface and Front End Design</b>                           | <b>47</b> |
| 6.1      | User Interface Design Goals and Visual Identity . . . . .            | 47        |
| 6.1.1    | Integration as a Micro Front-End . . . . .                           | 47        |
| 6.1.2    | Interaction Model . . . . .                                          | 48        |
| 6.1.3    | Visual Design and Color Scheme . . . . .                             | 48        |
| 6.1.4    | Clarity, Trust, and Transparency . . . . .                           | 48        |
| 6.2      | Technology Stack . . . . .                                           | 49        |
| 6.2.1    | Core Framework & Language . . . . .                                  | 49        |
| 6.2.2    | UI Component Library . . . . .                                       | 50        |
| 6.2.3    | State & Data Management . . . . .                                    | 50        |
| 6.2.4    | Routing . . . . .                                                    | 51        |
| 6.2.5    | Form Handling . . . . .                                              | 51        |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| 6.3      | Core Interface Components . . . . .        | 52        |
| <b>7</b> | <b>Technology Stack</b>                    | <b>59</b> |
| 7.1      | Overview of the Technology Stack . . . . . | 59        |
| 7.2      | Project Structure . . . . .                | 61        |
| 7.2.1    | scrap_data.py . . . . .                    | 62        |
| 7.2.2    | build_vector.py . . . . .                  | 64        |
| 7.2.3    | app.py . . . . .                           | 64        |
| 7.3      | OpenAI - GPT . . . . .                     | 65        |
| 7.3.1    | text-embedding-3-small . . . . .           | 65        |
| 7.3.2    | gpt-4o-mini . . . . .                      | 66        |
| 7.3.3    | Cost . . . . .                             | 67        |
| <b>8</b> | <b>Conclusion</b>                          | <b>71</b> |
| 8.1      | Synopsis and Final Conclusions . . . . .   | 71        |
| 8.2      | Future Work . . . . .                      | 72        |
|          | <b>Bibliography</b>                        | <b>75</b> |
|          | <b>APPENDICES</b>                          | <b>79</b> |
| <b>A</b> | <b>Evaluation Data Set</b>                 | <b>81</b> |
| <b>B</b> | <b>GPT Tokens</b>                          | <b>85</b> |
| B.1      | Definition of Tokens . . . . .             | 85        |
| B.2      | Tokenization Process . . . . .             | 85        |
| B.3      | Tokens and Model Constraints . . . . .     | 86        |
| B.4      | Tokens and Cost . . . . .                  | 86        |

# List of figures

|     |                                                                             |    |
|-----|-----------------------------------------------------------------------------|----|
| 3.1 | System Architecture Flow . . . . .                                          | 12 |
| 5.1 | Comparison of retrieval performance across system configurations . . . . .  | 42 |
| 5.2 | Comparison of answer quality metrics across system configurations . . . . . | 43 |
| 7.1 | System Architecture . . . . .                                               | 61 |
| 7.2 | Sequence Diagram . . . . .                                                  | 62 |



# List of tables

|     |                                                                                           |    |
|-----|-------------------------------------------------------------------------------------------|----|
| 5.1 | Sample of evaluation questions and corresponding ground-truth URLs . . .                  | 38 |
| 5.2 | Average retrieval and answer quality metrics across all evaluation questions              | 41 |
| 5.3 | Average retrieval performance across all evaluation questions . . . . .                   | 41 |
| 5.4 | Average answer quality scores across all evaluation questions . . . . .                   | 42 |
| 6.1 | Core framework and language . . . . .                                                     | 49 |
| 6.2 | UI Component Libraries . . . . .                                                          | 50 |
| 6.3 | State and Data Management . . . . .                                                       | 50 |
| 6.4 | Routing . . . . .                                                                         | 51 |
| 6.5 | Form Handling Technologies . . . . .                                                      | 51 |
| 7.1 | Technology stack used in the implementation of the RAG-based university chatbot . . . . . | 60 |
| 7.2 | Average token usage per user query . . . . .                                              | 68 |
| A.1 | Complete evaluation question set with ground-truth URLs . . . . .                         | 81 |



# Abbreviations

|     |                                     |
|-----|-------------------------------------|
| UTH | University of Thessaly              |
| ECE | Electrical and Computer Engineering |
| RAG | Retrieval Augmented Generation      |
| LLM | Large Language Model                |
| AI  | Artificial Intelligence             |
| NLP | Natural Language Processing         |





# Chapter 1

## Introduction

University websites are essential for students, staff and potential applicants to get academic and administrative information. As universities grow in size and complexity, there is larger amount of information, creating large and varied information spaces. Even though these kinds of websites give a lot of information about how institutions work, users often have trouble finding the exact and useful information they need.

Traditional website navigation and searching based on keywords require users to be familiar with university structure and terminology. This limitation can be an obstacle for users to have effective information access, particularly for users formulating queries in natural language or unfamiliar with institutional conventions. As a result, there is a growing need for interfaces that are easier to use and can help people quickly find reliable information.

Recent improvements in natural language processing have made it possible to create conversational agents that can interact with users in natural language. While these systems can improve the user experience, using them in domains that require information accuracy, such as university websites, can be very challenging. In particular, generative language models may produce incomplete or inaccurate responses, when not provided with verified source material. This demonstrates the how challenging it is to make sure that conversational systems can reliably find and present information.

### 1.1 Paper Subject

This thesis examines the design and evaluation of an AI Chatbot with a Retrieval-Augmented Generation (RAG) architecture for the e-ce.uth.gr website. The goal of the proposed system

is to provide precise and relevant answers to user questions through the combination of information retrieval methodologies with natural language generation.

The work addresses three primary problems: the limited capability of keyword-based search systems to handle natural language queries, the risk of hallucination in standalone LLMs and the difficulty of retrieving relevant information from structurally complex web-based content. These problems are especially clear on university websites, where each page often covers more than one topic and user questions can cover more than one type of information.

To address these issues, the thesis proposes a RAG-based pipeline in which university website content is collected and divided using semantic chunking techniques. Each chunk represents a coherent unit of information and is embedded and indexed to support similarity-based retrieval. Metadata are associated with the indexed content to improve retrieval precision. The retrieved information is then given to a large language model as a base of knowledge, which uses it to make responses that are limited by the source material.

Through experimentation and trial and error, this thesis dives into how different retrieval and chunking strategies perform for retrieval precision and answer quality. The findings aim to provide insights into the design of reliable AI Chatbots for information systems.

### **1.1.1 Contribution**

The contribution of this thesis:

1. A study was done on Chatbots and Retrieval-Augmented Generation (RAG) architectures, emphasizing their relevance to information-dense websites.
2. Scraping official university pages and using semantic chunking to turn the extracted data into coherent information chunks.
3. A retrieval framework using dense vector embeddings, incorporating metadata-aware indexing to improve the relevance and precision.
4. Several retrieval and chunking strategies were applied and evaluated.
5. By combining the retrieval result with an LLM, a RAG-based chatbot system was made so that the answers it gives are based on information from reliable websites.

6. An evaluation was conducted utilizing user queries to assess both retrieval effectiveness and the quality of the generated responses.

## 1.2 Organization

The remainder of this thesis is organized as follows: Chapter 2 reviews related work in the areas of chatbots, information retrieval, and RAG architectures. Chapter 3 presents the implementation of the AI Chatbot, including data scraping, index embedding, and answer generation. Chapter 4 provides a comparative analysis of different retrieval architectures. Chapter 5 describes the evaluation methodology and discusses the results. Chapter 6 focuses on the design and implementation of the user interface and front-end components of the AI Chatbot. Chapter 7 presents the technology stack and tools used throughout the development of the system. Finally, Chapter 8 concludes the thesis and talks about what needs to be done as future work.



# Chapter 2

## Literature Review

### 2.1 Chatbots in Higher Education

Colleges and universities use web-based information systems to share information about admissions, course registration, scholarships and student services. As the amount and complexity of this information keep growing, students often have to go through many different web pages and interfaces to find what they need, which makes things less efficient and makes users less happy [1]. The growing number of students and academic programs makes this problem even worse because it means more work for administrative staff.

To solve these problems, researchers have looked into using conversational agents, also known as chatbots, as a different way to get university information. Chatbots are software systems that use AI to make conversations between people sound like they are talking to each other in real life. This lets people get information by talking to each other instead of using a keyword-based search like they usually do [2]. Chatbots have been around for a long time, but they aren't used much in higher education yet. A lot of schools still use static FAQs or rule-based systems.

Previous research points out many advantages of implementing chatbots in academic websites. Chatbots can offer 24/7 assistance, give consistent answers, and quickly handle common questions about admissions and academic advice. This lightens the load on administrative staff [3]. Adding chatbots to university websites has also been shown to make them more accessible, easier to use and more engaging by giving students immediate help and guiding them through university processes [4].

Current research, however, also highlights problems with using chatbots, despite said

benefits. A major limitation is ensuring accuracy and relevance, particularly when dealing with complex and nuanced queries [1]. Many systems use rule-based logic or simple machine learning models, which makes it hard for them to grow as content grows and to handle a wide range of user questions. These problems show that there is a need for better chatbot architectures that are based on actual facts and are able to handle lots of different types of information.

## 2.2 Large Language Models (LLMs)

Large Language Models (LLMs) have recently shown big improvements in many natural language processing tasks, such as text generation, summarization, information retrieval, and conversational interaction. Most of the time, these models use transformer architectures and are trained on large data sets to predict and create coherent text sequences [5]. Their ability to generalize across various tasks lead to a fast rise of both research and applications, establishing LLMs as a fundamental element of modern AI systems.

People are becoming more interested in LLMs because they can be used as general-purpose language interfaces. This means that machines can do hard tasks with little extra engineering.. Reinforcement Learning from Human Feedback (RLHF) and other new training methods have made LLM outputs even easier to use and more in line with what people want by adding human preferences to the training process [6]. As a result, LLMs are being used more and more in software systems through cloud-based APIs or open-source deployments, which makes them useful in many different areas [7].

Recent research has begun to regard LLMs not just as development tools but also as software components integrated within larger systems. From this point of view, applications can use LLMs as functional parts that are responsible for reasoning instead of seeing them as extra tools [8]. This change has made it possible to quickly build complex applications that would have taken a lot of manual coding before, especially in systems that users talk to.

Even though LLMs are very powerful, they have some basic flaws that make it hard to use them directly in areas where information plays an important role. One of the biggest problems is that LLMs often make up content that sounds good but isn't true or can't be checked [6]. Because LLMs make text based on learned statistical patterns instead of explicit knowledge grounding, they may give out wrong information with a lot of confidence.

One big problem is that there is no source. Because LLMs don't give citations or proof for the claims they make [5], it's harder to check how accurate they are. Because of this lack of openness, LLMs are not reliable and can't be used in situations where truth and accuracy are important. LLMs are good at comprehending natural language, but they don't always get it right. That's why it's important to check the answers against other sources of information to make sure they are true, can be checked, and are trustworthy.

## 2.3 Information Retrieval

Information Retrieval (IR) is a key part of information systems. It is the foundation for applications like search engines, question-answering systems and chatbots. Finding and retrieving documents that are relevant to a user's query is the primary goal of IR. This usually means ranking several possible answers based on how relevant they are [9]. Since user queries are often vague or not specific enough, good retrieval systems need to find a balance between speed, relevance, and understanding the meaning of the search.

Early information retrieval systems primarily utilized term-based retrieval methods, which match requests and documents through word similarity. These methods are efficient and scalable, but they have built-in problems like synonymy, polysemy, and lexical mismatch that make it hard for them to understand what users mean when they search for something [9]. Many people still use traditional models like BM25 as first-stage retrievers because they are very good at finding relevant documents in large corpora.

Advances in natural language processing and the availability of large datasets have led to the adoption of neural and deep learning techniques for IR. Dense retrieval techniques present queries and documents as continuous vector embeddings, allowing semantic matching. These methods, which are often based on word embeddings or transformer architectures, have shown that they are better at figuring out what words mean in context and what the user wants, especially for natural language queries [9]. But dense retrieval models often require a lot of computing power and large training datasets, which can make them hard to use in the real world.

Most modern IR systems use a multi-stage architecture with a retrieval phase at the beginning and a ranking or re-ranking phase after that. In this framework, an effective first-stage retriever selects a group of documents that may be useful, and a second-stage model that uses

more processing power improves the ranking to make the results better [9]. This separation lets systems stay scalable while making retrieval more effective. It also serves as the basis for many modern search and question-answering systems.

Hybrid retrieval methods became a useful alternative to deal with the pros and cons of both lexical and semantic approaches. By combining sparse, term-based representations with dense semantic embeddings, hybrid systems improve recall and precision for different types of queries. It has been shown that hybrid retrieval methods that use both exact keyword matching and semantic similarity work better than methods that only use lexical or dense approaches [9]. These strategies are particularly pertinent for domains marked by heterogeneous content, where queries may include both precise factual terminology and broader semantic intentions.

## 2.4 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) has become a leading approach to overcoming the shortcomings of large language models in tasks that require a lot of knowledge. LLMs are good at comprehending and generating natural language, but they often hallucinate and don't have strong connections to sources. RAG solves these issues by integrating information retrieval into the generation process, allowing models to base their outputs on relevant retrieved evidence instead of depending exclusively on internal parametric knowledge [10, 11].

A RAG system has two main parts: a retriever and a generator. When a user asks a question, the retriever searches an external collection to find documents or passages that are relevant. These are then used in the generation process to create the final answer [11]. RAG systems can improve factual accuracy, make them more robust, and make them more transparent with this modular design, all without having to retrain the underlying language model, which is expensive [12].

A retrieve-and-generate pipeline, in which retrieval is carried out once before generation, is commonly used in early RAG techniques. In this configuration, the language model generates an answer based on both the original query and the content that was retrieved. This static retrieval approach can be restrictive for complex or long-form generation tasks, where pertinent data may need to be gathered iteratively during the generation process, even though it works well for short-form question answering [10]. To address this limitation, recent work



has explored more dynamic retrieval strategies, where models actively decide when and what information to retrieve as generation progresses.

RAG methods can also be categorized according to how retrieved information is incorporated into the generator. The most common way to do this is to add the retrieved text to the user's query, which gives the generator clear context-specific grounding. Alternative methods combine retrieval results as latent representations or directly affect the decoding process by adding logit-level information [11]. Even though these methods differ in how hard they are to use and how much they cost to run, input-level augmentation is still the most popular because it is easy to use and works with transformer-based generators that are already in use.

From a retrieval perspective, RAG systems may use sparse, dense, or hybrid retrieval methods to find relevant data. Sparse methods such as BM25 rely on word matching and are efficient for exact term queries, whereas dense retrieval methods use embedding-based similarity to represent semantic relevance. Hybrid retrieval strategies merge these two methodologies to enhance recall, rendering them especially effective for practical scenarios involving diverse data [11, 12].

RAG is a good and scalable way to use language models in areas where information is very important. RAG systems let developers update or add to knowledge bases without changing the model. This makes facts more accurate and cuts down on hallucinations. These qualities make RAG a great choice for university information systems and other places where it's important to have reliable, open, and wide coverage of large document collections.



## Chapter 3

# Implementation

This chapter presents the overall system architecture of the Retrieval-Augmented Generation chatbot that was developed for the University of Thessaly’s Department of Electrical and Computer Engineering website. The architecture is built to help people give accurate, grounded, and appropriate answers by combining modern neural retrieval and generation techniques with structured web data processing. There are two main parts to it: an offline data preparation pipeline that scrapes, cleans, semantically structures, and indexes the website content, and an online query processing pipeline that does hybrid retrieval, re-ranking, and grounded answer generation while the program is running. Figure X shows the whole system architecture and how its parts work together.

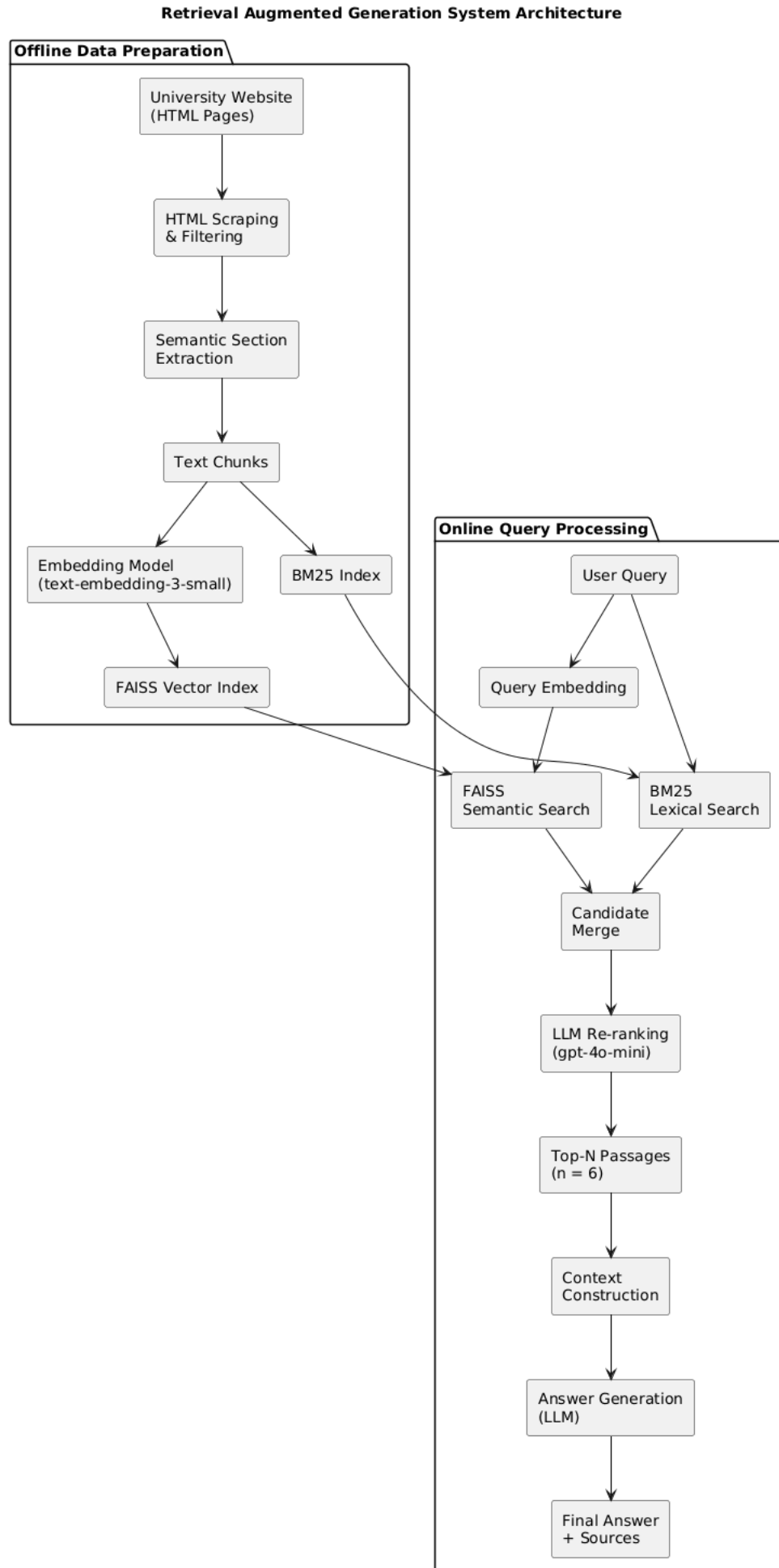


Figure 3.1: System Architecture Flow

## 3.1 Data Collection

### 3.1.1 Website Scraping Process

The data source for the AI Chatbot is of course the official ECE website ([e-ce.uth.gr](https://www.e-ce.uth.gr)), which contains publicly available information intended for prospective and current students. Since this information is distributed across multiple webpages in HTML format, we need to scrap those pages, clean it of HTML elements such as navigation menus, scripts, and styling artifacts, filter out the information meaningful to us and reformat it into useable text chunks.

Starting point is to collect all the URLs of the xml sitemap of the website (<https://www.e-ce.uth.gr/sitemap.xml>). An XML sitemap is a file, written in Extensible Markup Language (XML), that lists all the URLs on a website for search engines to discover, crawl, and index more efficiently, acting as a roadmap to guide crawlers to all the content. ECE's XML sitemap contains 6 sitemaps:

- <https://www.e-ce.uth.gr/sitemap-misc.xml>: Contains the front page of the website and the XML sitemap URL
- <https://www.e-ce.uth.gr/category-sitemap.xml>: Contains the pages that have links to announcements
- <https://www.e-ce.uth.gr/post-sitemap.xml>: Contains the actual URLs of all the announcements
- <https://www.e-ce.uth.gr/page-sitemap.xml>: Contains the URLs of all the pages we need like studies, courses, faculty
- <https://www.e-ce.uth.gr/page-sitemap2.xml>: Continuation page

The announcements XML is intentionally excluded from the scraping process due to its highly dynamic nature and large size because it contains all the content of the announcements of the last at least 10 years. Thus making this feed primarily is less suitable for static indexing and would significantly increase the volume of data without proportionate benefits to the chatbot's intended use cases.

### 3.1.2 Semantic Chunking Strategy

To achieve effective retrieval in the proposed RAG AI chatbot, the scraped HTML content must be divided into semantically meaningful blocks rather than arbitrary text blocks. Traditionally, this could happen just splitting the text from the website into predefined sized chunks. However this way, the data will be unstructured and we will not be able to retrieve the relevant information in its entirety. For this purpose, we implemented a semantic section extraction procedure in order to leverage the structural cues present in HTML documents.

The semantic extraction process traverses the raw HTML to find content boundaries. In particular, HTML heading elements (`<h1>`, `<h2>`, and `<h3>`) are indicators of topic shifts and we are going to use them to define the start of new semantic sections. Each section has a heading that reflects the topic of the content that follows.

At the beginning of the process, we retrieve the page title and use it as the default heading for the initial section. As we go through the HTML body, we collect content contained within paragraph (`<p>`) and list item (`<li>`) elements and assign them to each section. This approach ensures that only semantically relevant and human-readable text is scrapped, while other HTML elements are ignored.

ECE's website uses lots of tables to display data, such as information about a course or a professor. It's very important to make sure that we correctly scrap those tables, so that we don't lose valuable data. When we encounter a table element, we extract its rows and cells and convert them into a text representation by concatenating cells values using `<br>`. This allows information on tables to be saved in a linear text format that can be interpreted by embedding and language models.

Whenever we encounter a new heading element, the content collected so far is finalized and we store it as an individual semantic section, and we move to a new section. The result of this process is a set of semantically coherent sections, with their corresponding heading and page title.

This heading-based segmentation technique generates text segments that are appropriate for embedding-based retrieval and closely match the webpage's logical structure. The method increases retrieval accuracy and lessens the possibility that the language model will receive fragmented or irrelevant context during answer generation by guaranteeing that each chunk represents a single topical unit.

## 3.2 Embedding and Indexing

### 3.2.1 Embedding and Vector Representation

To enable semantic retrieval over the extracted text chunks, We use a text embedding model to convert each extracted text chunk into a numerical vector. Semantically similar texts are placed closer together in a high-dimensional vector space created by text embeddings. This representation makes it possible to effectively use vector distance metrics, like cosine similarity, for similarity-based retrieval.

In the proposed system, we decided to use the "text-embedding-3-small" model by OpenAI to generate vector embeddings for each semantic text chunk. This model generates fixed-length dense vector representations that capture the input text's meaning. "Text-embedding-3-small" provides a good trade-off between embedding quality, storage needs, and indexing performance when compared to larger embedding models, which makes it ideal for medium-sized document collections like university websites.

#### Mathematical Formulation of Text Embeddings

Let  $\mathcal{T}$  denote the space of all possible text sequences and let  $\mathbb{R}^d$  represent a  $d$ -dimensional real-valued vector space. A text embedding model can be formally described as a function:

$$f : \mathcal{T} \rightarrow \mathbb{R}^d \quad (3.1)$$

where  $f(t)$  maps an input text sequence  $t \in \mathcal{T}$  to a dense vector representation  $\mathbf{e} \in \mathbb{R}^d$ . The objective of the embedding function is to preserve semantic similarity, such that semantically related texts are mapped to nearby points in the embedding space.

#### Tokenization and Input Representation

Given an input text sequence  $t$ , it is first tokenized into a sequence of tokens:

$$t = (w_1, w_2, \dots, w_n) \quad (3.2)$$

Each token  $w_i$  is mapped to a token embedding vector  $\mathbf{x}_i \in \mathbb{R}^k$ , where  $k$  denotes the dimensionality of the token embedding space. To preserve positional information, positional encodings  $\mathbf{p}_i$  are added:

$$\mathbf{z}_i = \mathbf{x}_i + \mathbf{p}_i \quad (3.3)$$

The resulting sequence  $\{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n\}$  is provided as input to the transformer encoder.

### Transformer-Based Encoding

The embedding model employs a transformer encoder architecture composed of stacked self-attention layers. Within each layer, self-attention is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left( \frac{QK^\top}{\sqrt{d_k}} \right) V \quad (3.4)$$

where  $Q$ ,  $K$ , and  $V$  are the query, key, and value matrices derived from the input representations, and  $d_k$  is the dimensionality of the key vectors. Through multiple layers of self-attention and feed-forward transformations, the model produces contextualized token representations:

$$(\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n) \quad (3.5)$$

where each  $\mathbf{h}_i$  encodes contextual information about token  $w_i$  relative to the entire sequence.

### Sequence-Level Embedding

To obtain a fixed-length representation for the entire text sequence, a pooling operation is applied over the token-level representations:

$$\mathbf{e} = \text{Pool}(\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_n) \quad (3.6)$$

where  $\text{Pool}(\cdot)$  may represent mean pooling, attention-based pooling, or selection of a designated sequence-level token. The resulting vector  $\mathbf{e} \in \mathbb{R}^d$  constitutes the semantic embedding of the input text.

### Semantic Similarity

To measure semantic similarity between two embedded texts  $\mathbf{e}_a$  and  $\mathbf{e}_b$ , cosine similarity is employed:



$$\text{sim}(\mathbf{e}_a, \mathbf{e}_b) = \frac{\mathbf{e}_a \cdot \mathbf{e}_b}{\|\mathbf{e}_a\| \|\mathbf{e}_b\|} \quad (3.7)$$

Higher similarity values indicate greater semantic relatedness. This metric enables nearest-neighbor retrieval in the embedding space.

### Training Objective (Conceptual)

Although the internal training procedure of “text-embedding-3-small” is unknown, such models are typically trained using contrastive learning objectives. Given a positive text pair  $(t_i, t_j)$  and a set of negative examples  $(t_i, t_k)$ , the loss function can be expressed as:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(f(t_i), f(t_j))/\tau)}{\sum_k \exp(\text{sim}(f(t_i), f(t_k))/\tau)} \quad (3.8)$$

where  $\tau$  is a temperature parameter controlling similarity scaling. This objective encourages semantically related texts to be embedded closer together while pushing unrelated texts farther apart.

### Relevance to Retrieval-Augmented Generation

Information retrieval becomes a vector space similarity maximization problem when both data chunks and user queries are embedded using the same function  $f$ . The Retrieval-Augmented Generation architecture used in this work is based on this mathematical framework, which permits effective and semantically sound retrieval.

## 3.2.2 Vector Indexing Using FAISS

After generating dense vector embeddings for each text chunk, what we need to do is create an index to support similarity-based retrieval. Let  $\{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N\}$  denote the set of embedding vectors, where each  $\mathbf{e}_i \in \mathbb{R}^d$  and  $d$  is the embedding dimensionality. We will use the Facebook AI Similarity Search (FAISS) library to store and index these embeddings.

Before indexing, all embedding vectors are normalized to unit length:

$$\mathbf{e}_i \leftarrow \frac{\mathbf{e}_i}{\|\mathbf{e}_i\|} \quad (3.9)$$

This normalization step helps for cosine similarity to be computed using the inner product between vectors. Specifically, for two normalized vectors  $\mathbf{q}$  and  $\mathbf{e}_i$ , cosine similarity is calculated by:

$$\text{sim}(\mathbf{q}, \mathbf{e}_i) = \mathbf{q} \cdot \mathbf{e}_i \quad (3.10)$$

Following normalization, we feed the embeddings into a FAISS `IndexFlatIP` index, which performs nearest-neighbor search using inner product similarity. This indexing method does not rely on approximation techniques and therefore guarantees retrieval of the true top- $k$  most similar vectors, making it suitable for our use case.

Given a query embedding  $\mathbf{q} \in \mathbb{R}^d$ , the index retrieves a ranked list of embeddings  $\{\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_k}\}$  such that:

$$\mathbf{q} \cdot \mathbf{e}_{i_1} \geq \mathbf{q} \cdot \mathbf{e}_{i_2} \geq \dots \geq \mathbf{q} \cdot \mathbf{e}_{i_k} \quad (3.11)$$

The resulting top- $k$  vectors match to the semantic text chunks that are most similar to the user query. These retrieved chunks are feeded to the next component of the Retrieval-Augmented Generation pipeline, that we will see in the following section.

The FAISS index that we have constructed will be saved, so that we can easily reuse them in all the incoming requests, avoiding the need to rebuild the index for each session. This design choice improves response time, saves token usage, thus money spent on OpenAI models and supports scalable deployment.

### 3.2.3 Lexical Retrieval Using BM25

Dense vector embeddings make semantic similarity-based retrieval possible, but they might perform poorly for queries that depend on precise lexical matching, like course name, professors, acronyms or specific terminology. To overcome this, we will incorporate the BM25 ranking function.

BM25 (Best Matching 25) is a probabilistic information retrieval model that ranks documents based on term frequency and inverse document frequency while applying length normalization. Unlike embedding-based retrieval, BM25 is very good when exact keywords appear between the query and the document because it operates directly on tokenized text.

At the application startup, a BM25 index is constructed on all semantic chunks. Let

$\mathcal{D} = \{d_1, d_2, \dots, d_N\}$  denote the set of the extracted text chunks. Each document  $d_i$  is first normalized by converting all text to lowercase and tokenized into a sequence of terms:

$$d_i = (w_{i1}, w_{i2}, \dots, w_{im}) \quad (3.12)$$

This preprocessing ensures that there is consistency on lexical matching and reduces sparsity caused by capitalization. The tokenized documents are then used to create a BM25 index, which stores statistics, like term frequency and document frequency, required for ranking.

The BM25 relevance score between a query  $q$  and a document  $d_i$  is calculated as

$$\text{BM25}(q, d_i) = \sum_{w \in q} \text{IDF}(w) \cdot \frac{f(w, d_i) \cdot (k_1 + 1)}{f(w, d_i) + k_1 \left(1 - b + b \cdot \frac{|d_i|}{\text{avgdl}}\right)} \quad (3.13)$$

where:

- $f(w, d_i)$  is the frequency of term  $w$  in document  $d_i$ ,
- $|d_i|$  is the length of document  $d_i$ ,
- $\text{avgdl}$  is the average document length in the corpus,
- $k_1$  and  $b$  are hyperparameters controlling term frequency saturation and length normalization.

By creating the BM25 index at application start up, the retrieval mechanism can efficiently score and rank documents without additional processing overhead. The BM25 index serves as a complementary retrieval mechanism to dense vector search (FAISS index), enabling the system to handle both semantically phrased queries and keyword-focused queries.

### 3.3 Hybrid Query Logic

Given a user question  $q$ , the hybrid retrieval process consists of three main stages: vector-based retrieval (FAISS index), lexical retrieval (BM25), and candidate merging, to find the top 20 most relevant documents.

#### 3.3.1 Dense Vector Retrieval

Firstly, the user question  $q$  is first transformed into a dense vector representation using the same "text-embedding-3-small" embedding model  $f$  applied during indexing. Let:

$$\mathbf{q} = f(q) \in \mathbb{R}^d \quad (3.14)$$

denote the resulting query embedding. The vector is normalized to unit length to enable cosine similarity computation via inner product:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|} \quad (3.15)$$

Afterwards, using the FAISS index, we will find the  $k$  nearest neighbors (in our system, it is decided to  $k = 20$ ), which means the top- $k$  embeddings most similar to  $\mathbf{q}$ . The set of vector-based candidates is defined as:

$$\mathcal{C}_{\text{vec}} = \{i_1, i_2, \dots, i_k\} \quad \text{such that} \quad \mathbf{q} \cdot \mathbf{e}_{i_1} \geq \dots \geq \mathbf{q} \cdot \mathbf{e}_{i_k} \quad (3.16)$$

where  $\mathbf{e}_i$  denotes the embedding of the  $i$ -th text chunk.

### 3.3.2 Lexical BM25 Retrieval

Afterwards, the same query  $q$  is processed using the BM25 lexical retrieval model. The query is lowercased and tokenized into a sequence of terms:

$$q = (w_1, w_2, \dots, w_m) \quad (3.17)$$

BM25 relevance scores are calculated between the query and all indexed text chunks. The top- $k$  highest-scoring documents are selected, which yields this set of candidates:

$$\mathcal{C}_{\text{bm25}} = \{j_1, j_2, \dots, j_k\} \quad (3.18)$$

This is very effective for queries involving exact terminology, such as course codes or names of professors.

### 3.3.3 Candidate Set Fusion

To make the final set of retrieval candidates, the results of vector-based and BM25-based retrievals are merged by taking the union of their respective sets:

$$\mathcal{C}_{\text{hybrid}} = \mathcal{C}_{\text{vec}} \cup \mathcal{C}_{\text{bm25}} \quad (3.19)$$

By using this fusion strategy, documents that are retrieved based on lexical relevance or semantic similarity are kept for further processing. The hybrid approach minimizes the risk of losing potentially important information while maximizing recall by delaying final ranking to a later stage.

All things considered, the system can handle both natural-language and keyword-driven queries thanks to this hybrid retrieval process than it could with either technique alone. Then, the candidate set candidate set  $\mathcal{C}_{\text{hybrid}}$  is given to a re-ranking system.

## 3.4 Reranking

### 3.4.1 LLM-Based Relevance Scoring Using GPT-4o-mini

To perform re-ranking of the candidate documents based on their relevance, we utilize a *lightweight LLM*, **GPT-4o-mini**, to perform semantic relevance evaluation. Because the model can reason over the relationship between the user query and the candidate passage, using an LLM for re-ranking allows for a more nuanced evaluation of passage usefulness than similarity-based scores alone.

GPT-4o-mini was chosen because it offers a good trade-off between computational efficiency, reasoning power and token cost. The re-ranking task requires consistent, low-latency evaluations across several candidate passages. Even though larger models (like GPT-4o, GPT-5o) might provide bigger improvements in semantic judgment, GPT-4o-mini minimizes token cost and response time while offering adequate semantic understanding for this task.

### 3.4.2 Prompt Engineering

We will frame this relevance scoring task as a constrained numerical evaluation problem. We will give the LLM the passage text and the user question for every candidate passage, and prompt it to produce a single numerical score. Below is the prompt that was used for this:

**Question:**

{question}

**Passage:**

{passage\_text}

**Instruction:**

Score how well the passage answers the question on a scale from 0 to 5. Return only the number.

This prompt makes it clear that the model output must be a number, which makes it less confusing and easier to parse later on. The system reduces the chance of inconsistent or long-winded answers during scoring by not allowing open-ended text generation.

### 3.4.3 Prompt Engineering Considerations

Several prompt engineering principles guided the design of the relevance scoring prompt:

- **Explicit task framing:** The prompt clearly distinguishes between the question and the document, enabling the model to reason about their relationship.
- **Discrete scoring scale:** A fixed scale from 0 to 5 provides sufficient level of detail for ranking while maintaining interpretability.
- **Output constraint:** Instructing the model to return only a number ensures compatibility with automated ranking, when parsing the number on the application code from the models response.
- **Deterministic behavior:** The model is queried with a temperature of zero, which means giving the model less creativity with its answer, reduces randomness and improves reproducibility of scores.

### 3.4.4 Integration into the Retrieval Pipeline

The language model gives each candidate passage  $c_i$  a relevance score  $s_i$  that ranges from 0 to 5. These scores are only used to rank things and don't have any effect on the content of the answer that is generated. The answer generation component gets the **top 6** ranked passages as input.

The system stays modular by decoupling relevance estimation from answer generation. This means that the re-ranking component can be improved or replaced on its own in the future.

In general, using GPT-4o-mini as a re-ranking model lets you choose passages with precision and context awareness while keeping the Retrieval-Augmented Generation framework efficient and scalable.

## 3.5 Answer Generation

The system moves on to the answer generation phase after the re-ranking stage. This stage is in charge of putting together a natural language answer to the user’s question using only the most relevant passages that were found. A grounded generation approach is used to make sure that the facts are correct and that hallucinations don’t happen.

### 3.5.1 Context Construction

Let  $\mathcal{C}_{\text{final}} = \{c_1, c_2, \dots, c_n\}$  denote the set of top-ranked passages selected during the re-ranking stage, where  $n = 6$  in this work. These passages are put together to make one contextual input for the LLM:

$$\text{Context} = \bigoplus_{i=1}^n c_i \quad (3.20)$$

where  $\oplus$  means joining strings together. The context only includes the raw text of each document; it does not show the LLM any relevance scores or similarity distances. This choice makes sure that the model’s response isn’t biased by retrieval metadata and that generation is only based on facts.

Each passage is clearly marked as a source segment, which lets the model treat the context given as reliable proof.

### 3.5.2 Answer Generation Prompt

The task of generating answers is set up as a problem with limited instructions. The language model is told to act as a virtual assistant for the University of Thessaly’s Department of Electrical and Computer Engineering and to only use the information given to it to answer questions.

The prompt used for answer synthesis is shown below:

**Είσαι ένας βοηθός για τον ιστότοπο του Τμήματος Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών του Πανεπιστημίου Θεσσαλίας.**

Απάντησε στην ερώτηση χρησιμοποιώντας ΑΠΟΚΛΕΙΣΤΙΚΑ τις πληροφορίες που δίνονται παρακάτω. Αν η απάντηση δεν προκύπτει ξεκάθαρα από το περιεχόμενο, πες ότι η πληροφορία δεν είναι διαθέσιμη στον ιστότοπο του Τμήματος.

**Ερώτηση:** {question}

**Πληροφορίες:** {context}

**Οδηγίες:**

- Απάντησε στα Ελληνικά
- Να είσαι σαφής και περιεκτικός
- Μην χρησιμοποιήσεις αστερίσκους ή markdown
- Μην προτρέπεις τον χρήστη να ψάξει αλλού

This prompt limits how the model generates responses very strictly. This makes sure that the responses stay true to the content that was retrieved and meet the style requirements of an official university information system.

### 3.5.3 Prompt Engineering

Several prompt engineering principles were used to improve answer quality and reliability:

- **Role framing:** The instruction that says the model is an assistant for the website of the University of Thessaly's Department of Electrical and Computer Engineering is a way to frame the role. This puts the model's behavior in context, limiting its responses to an institutional point of view and helping it use the right academic tone, vocabulary, and level of formality for an official university information service.
- **Exclusive grounding:** The model is told very clearly to only base its answer on the passages that were given. The generation process is based only on retrieved evidence because it doesn't allow the use of outside or prior knowledge. This greatly lowers the risk of hallucination and makes sure that all responses can be traced back to the university's official web content.



- **Explicit fallback behavior:** The model is told to admit that the requested information is missing or not clearly supported by the retrieved content instead of guessing. This limit stops hallucinated answers and makes the system more of a reliable information assistant than a generative oracle.
- **Language control:** Responses are constrained to Greek to match the target audience and source content. Moreso, the target users, the source content, and the responses are all in Greek, so the prompt instructions are also in Greek. This alignment cuts down on unnecessary language switching in the model, which makes it more grounded in the source text, keeps formal academic phrasing, and gives more consistent and reliable answers.
- **Stylistic constraints:** To make sure that responses come back as plain text, the model's output explicitly forbids Markdown and other formatting artifacts. This lets the system format, style, and post-process the content before it is shown, which makes sure that the website looks the same everywhere and that model-generated markup doesn't cause rendering problems.

### 3.5.4 Model Configuration

**GPT-4o-mini** with a temperature of zero is used to compile the answers. This setup encourages outputs that are predictable and can be repeated, which is especially important for informational systems where consistency and reliability are more important than creative variation.

The generation model is called only once for each user query, which cuts down on latency and processing costs. The final answer comes with the URLs of the source passages that were used to build the context. This makes the response clear and easy to follow.

In general, this answer generation strategy makes sure that the chatbot gives correct, context-aware, and institutionally appropriate answers, which is what you would expect from a Retrieval Augmented Generation system used in an academic setting.



## Chapter 4

# Comparative Analysis of Retrieval Architectures

This chapter showcases a comparison of three retrieval architectures developed during the implementation of the chatbot. Each implementation is based on a different set of design decisions about how to organize documents, how to scrap, and how semantic structure fits into Retrieval Augmented Generation (RAG) systems. The goal of this chapter is to take a look at how these architectural choices affect the accuracy, reliability, and overall quality of the answers.

The system began with a basic single-index baseline and fixed-size chunking. It was then improved step by step with a category-based multi-index approach and finally a hybrid metadata-enriched architecture. While the design change aimed to improve retrieval precision through content separation, empirical evaluation revealed unexpected degradation in performance. This chapter examines the causes of these underwhelming results and explains how the final hybrid approach resolves the problems observed in previous designs.

### 4.1 One Index - Predefined Chunking

The first version is a baseline Retrieval Augmented Generation (RAG) system. In this method, all the documents taken from the university website are processed in the same way, split into chunks using a fixed token-based method, put into a single vector index, and only found by searching for dense vector similarity. There is no explicit consideration of semantic structure or document hierarchy during chunking or retrieval.

### 4.1.1 Content Extraction and Preprocessing

The first step is to process each webpage to get its text content out of raw HTML. The extraction pipeline uses two different methods:

- **Main text extraction:** An automated text extraction tool pulls out the most readable parts of the text, focusing on accuracy and leaving out links, tables, and formatting artifacts.
- **Structural enrichment:** An HTML parser extracts extra structural elements like page titles, section headings (h1–h3), and tabular content.

The final text representation of a webpage is made by putting together the title, headings, main body text, and linearized table contents into one document:

$$D = T_{\text{title}} \parallel T_{\text{headings}} \parallel T_{\text{body}} \parallel T_{\text{tables}}$$

where  $\parallel$  denotes string concatenation. The document is then passed to the chunking stage.

### 4.1.2 Fixed-Size Token-Based Chunking

Document chunking is performed using a fixed-size, token-based strategy. Each document  $D$  is split into a sequence of chunks  $\{c_1, c_2, \dots, c_n\}$  such that each chunk contains at most  $M$  tokens, where  $M = 400$ .

A fixed-size, token-based method is used to chunk documents. We break each document  $D$  into a series of chunks  $\{c_1, c_2, \dots, c_n\}$ , with each chunk having no more than  $M$  tokens.  $M$  is 400. Chunks are constructed sequentially such that:

$$\sum_{w_j \in c_i} \tau(w_j) \leq M$$

A new chunk is made when adding a new word would make the token limit too high. This process goes on until the whole document is broken up.

This chunking strategy is *agnostic to semantic boundaries*, which is important. There are no section headings, topic changes, or logical document structure, which can lead to chunks that have parts of sections or topics that don't relate to each other.

### 4.1.3 Embedding and Index Construction

All generated chunks from all webpages are embedded into a shared vector space using a dense embedding model. Let  $c_i$  denote a chunk of text, and let the embedding function be defined as:

$$\mathbf{e}_i = f_{\text{embed}}(c_i) \in \mathbb{R}^d$$

where  $d$  is the embedding dimensionality.

The set of all chunk embeddings  $\{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N\}$  is stored in a single vector index. No metadata-based partitioning or filtering is applied; all chunks are treated as equally relevant candidates during retrieval.

### 4.1.4 Query Processing and Retrieval

Given a user query  $q$ , the system calculates its embedding:

$$\mathbf{q} = f_{\text{embed}}(q)$$

A nearest-neighbor search over the vector index is used to find the information. The top- $k$  chunks are chosen based on how similar their vectors are, which is a way to measure distance:  $\delta(\mathbf{q}, \mathbf{e}_i)$ :

$$\mathcal{R}_k = \arg \min_i \delta(\mathbf{q}, \mathbf{e}_i), \quad |\mathcal{R}_k| = k$$

In this implementation,  $k = 5$ . The retrieved chunks are put in order of the distance between them and then combined to make the context that is sent to the language model to make an answer.

### 4.1.5 Characteristics and Limitations

This baseline method has a simple and fast retrieval pipeline that doesn't need much preprocessing and doesn't depend on how the document is structured. But it does have some problems, such as:

- Chunks that don't make sense semantically
- Possible mixing of unrelated topics in one chunk
- Less accurate results for searches that are looking for specific sections, like deadlines or eligibility criteria

- No way to take advantage of known document categories or metadata

Even with these problems, this strategy gives us a good starting point for testing more advanced chunking and retrieval methods.

## 4.2 Category-Based Multi-Index Retrieval Strategy

The second implementation adds clear document categorization and multiple independent vector indices to the basic retrieval system. Instead of putting all the content into one shared index, the scraped pages are put into predefined semantic groups, and each group has its own embedding space and retrieval index. When a user asks a question, the system first puts it into one of these groups and then only searches the index that goes with that group.

### 4.2.1 Design Motivation

This method is meant to make retrieval more accurate by narrowing the search space and making sure that different types of content are kept separate. The system's goal is to get more focused and category-specific information than the single-index baseline by only searching a semantically relevant subset of documents.

This design, however, adds another requirement for correct query classification, which is looked at more closely in the next chapters on evaluation and discussion.

### 4.2.2 Data Collection via Sitemap Parsing

The department website's official XML sitemaps are used to obtain the content. There are two different sources for the sitemap:

- **Announcements sitemap:** <https://www.e-ce.uth.gr/post-sitemap.xml>: Contains URLs corresponding to announcements.
- **Information sitemaps:** <https://www.e-ce.uth.gr/page-sitemap.xml>, <https://www.e-ce.uth.gr/page-sitemap2.xml>: Contains URLs corresponding to all the other categories.

Only URLs that have been updated in the last 365 days are kept in the announcements category. This makes sure that the information you get is still relevant. In formal terms, let

$U_a$  be the set of announcement URLs and  $t(u)$  be the time stamp of the last change to URL  $u$ . The filtered set is defined as:

$$U'_a = \{u \in U_a \mid t_{\text{now}} - t(u) \leq 365 \text{ days}\}$$

All remaining informational URLs are extracted without any time-based filtering.

### 4.2.3 Rule-Based URL Categorization

URLs are assigned to categories based on their URL prefixes. Let  $\mathcal{C} = \{c_1, c_2, \dots, c_m\}$  denote the set of categories:

$$\mathcal{C} = \{\text{faculty, schedule, courses, secretariat, research, announcements}\}$$

Each category  $c_i$  is associated with a set of canonical URL prefixes  $P_{c_i}$ . A URL  $u$  is assigned to category  $c_i$  if:

$$\exists p \in P_{c_i} \text{ such that } u \text{ starts with } p$$

The announcements category is handled separately using the filtered announcement sitemap URLs.

This process results in disjoint URL buckets:

$$U = \bigcup_{i=1}^m U_{c_i}, \quad U_{c_i} \cap U_{c_j} = \emptyset \quad \forall i \neq j$$

### 4.2.4 Chunking, Embedding, and Index Construction

For every category  $c_i$ , all of the related webpages are crawled and processed on their own. The extracted text is split up into chunks using the same fixed-size, token-based method that was used in the baseline implementation. During chunking, there are no semantic boundaries or document structure.

Each piece of data  $c_{ij}$  that belongs to category  $c_i$  has been embedded into a dense vector representation:

$$\mathbf{e}_{ij} = f_{\text{embed}}(c_{ij}) \in \mathbb{R}^d$$

A different FAISS index,  $\mathcal{I}_{c_i}$ , is made for each category using only the embeddings that belong to that category. Along with each index, there is metadata that has the source URLs.

### 4.2.5 Query Classification and Routing

When a user sends in a query  $q$ , the system first uses an LLM-based classifier to figure out which category fits best. The classifier finds a mapping based on the set of categories  $\mathcal{C}$ :

$$g(q) \rightarrow c_k \in \mathcal{C}$$

The classifier is told to give only one category label, which is then used to choose the right index  $\mathcal{I}_{c_k}$ .

This step in routing makes a choice: it only looks up the chosen category index for this query and ignores all the others.

### 4.2.6 Category-Specific Retrieval and Answer Generation

After choosing the category  $c_k$ , the query embedding  $\mathbf{q}$  is calculated:

$$\mathbf{q} = f_{\text{embed}}(q)$$

Then, retrieval is done only in the category-specific index  $\mathcal{I}_{c_k}$  using the nearest-neighbor search. The top- $k$  pieces are chosen:

$$\mathcal{R}_k^{(c_k)} = \arg \min_{c_{kj}} \delta(\mathbf{q}, \mathbf{e}_{kj})$$

The LLM uses the retrieved chunks as context to make the final answer and give the source URLs. All answers must use only the information that was found in the category.

### 4.2.7 Analysis of the Multi-Index Retrieval Strategy Performance

The category-based multi-index retrieval strategy was intended to enhance precision by narrowing the search space; however, empirical assessment demonstrated that this method significantly underperformed compared to both the baseline single-index system and the final hybrid architecture. This drop in performance is mostly due to problems with retrieval, not problems with the language model itself.

#### Retrieval as the Primary Bottleneck

In Retrieval Augmented Generation (RAG) systems, the quality of the answer is mostly based by the quality of the context that was found. A language model that is very good at what it does can't give accurate or true answers if it can't find the right information.



The following failure pattern was often seen across evaluation questions:

- Relevant information existed within the scraped website content.
- The retrieval stage failed to find the appropriate chunks.
- The LLM gave vague, incomplete, or even hallucinated answers because it didn't have enough grounding.

This observation supports the general conclusion that retrieval errors, as opposed to generation errors, represent the primary failure mode in RAG-based systems.

### Query Routing as a Single Point of Failure

One major drawback of the multi-index architecture is that it requires a separate step for routing queries. There is only one category for each user query, and retrieval only happens within that category's index.

Formally, given a query  $q$  and category classifier  $g$ , retrieval is constrained to:

$$\mathcal{R}(q) = \mathcal{I}_{g(q)}$$

If the classifier picks the wrong category,  $g(q) \neq c^*$ , where  $c^*$  is the category that has the right information, the retrieval result set is empty when it comes to the right answer. There is no fallback mechanism or cross-category retrieval with this method, unlike the single-index method.

This hard routing decision creates a fragile dependency, meaning that even if classification accuracy is high, a single mistake in classification can cause the whole retrieval process to fail.

### Reduced Semantic Recall in Smaller Indices

When you split content across multiple indices, dense vector similarity search doesn't work as well. To find meaningful nearest neighbors, embedding-based retrieval compares a query vector  $q$  to a big enough and varied set of document embeddings.

Let  $\mathcal{E}$  be the group of embeddings in an index. Over-segmentation lowers  $|\mathcal{E}|$ , which gives:

$$|\mathcal{E}_{c_i}| \ll |\mathcal{E}_{\text{global}}|$$

As a consequence:

- Semantic neighborhoods become sparse.
- Distance metrics become less discriminative.
- Most of the time, nearest neighbors aren't very relevant.

This effect was especially clear for categories with little content volume, where even queries that were sent to the right place returned generic or loosely related chunks.

### Cascading Error Effects

When you mix hard query routing, category mismatch, and less semantic recall, you get a cascading failure effect. Mistakes made early in the pipeline can't be fixed later on because the generation model can only work with the context that was retrieved.

So, even though the multi-index architecture sounds good in theory, it gives up robustness for strict modularity. These results prompted the shift to a hybrid retrieval strategy that maintains a cohesive embedding space while utilizing categorical information in a more subtle, metadata-driven approach.

## 4.3 Hybrid Metadata-Based Retrieval Strategy

The final system design uses a hybrid retrieval strategy that takes the best parts of the previous ones and fixes their problems. This method uses a single unified vector index with metadata-aware filtering and hybrid retrieval techniques instead of keeping multiple separate indices or only using unconstrained dense retrieval.

The main goal of this design is to keep semantic recall and robustness while also adding categorical information in a way that is soft and flexible.

### 4.3.1 Unified Index with Semantic Density Preservation

All of the document chunks taken from the university website are stored in a single global index and put into a single shared vector space. Let  $\mathcal{E} = \{\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N\}$  be the complete set of embeddings for all chunks on all pages and in all categories.

The system makes sure of the following by keeping a single index:

- High embedding density

- Well-formed semantic neighborhoods
- Improved nearest-neighbor discrimination

Retrieval is formally executed across the entire embedding set:

$$\mathcal{R}_k = \arg \min_{i \in \{1, \dots, N\}} \delta(\mathbf{q}, \mathbf{e}_i)$$

This design directly addresses the semantic sparsity problem seen in the multi-index approach, where category-specific indices often didn't have enough content to make similarity-based retrieval reliable.

### 4.3.2 Metadata-Enriched Chunk Representation

All of the chunks are stored in one unified index, but each chunk has extra information about where it came from in the source document. Instead of just storing this information as external metadata fields, the contextual cues are put directly into the text of each chunk before it is embedded.

Specifically, each chunk has:

- the title of the source page,
- the heading of the section from which the chunk was extracted,
- the source URL,
- the section-specific textual content.

By putting this contextual information right into the chunk text, the resulting vector representations show both the meaning and the structure of the text. This method keeps a single embedding space while letting the language model understand retrieved chunks with enough context.

This enhancement does not divide or limit the retrieval space, which is important. During a similarity search, all chunks are still valid. The embedded contextual cues make both retrieval relevance and answer generation better without needing to filter based on metadata at retrieval time.

### 4.3.3 Support for Multi-Faceted Queries

Because retrieval is not constrained to a single category, the hybrid approach naturally supports multi-faceted and ambiguous user queries. Queries involving multiple institutional aspects—such as admissions procedures, academic regulations, and administrative services—can retrieve complementary information from different sections of the website.

This property is particularly important in our use case, where user questions often do not necessarily reflect website structure.

### 4.3.4 Robustness and Design Advantages

The hybrid metadata-based architecture has a number of benefits over the previous versions:

- Getting rid of hard routing failures
- Improved semantic recall due to unified embedding space
- Flexible use of categorical information without fragmentation
- More resilient to vague or incomplete questions

# Chapter 5

## Evaluation

This chapter presents the experimental evaluation of the RAG chatbot systems that were talked about in the previous chapter. The goal of this evaluation is to find out how well different retrieval strategies and system designs help users get accurate, faithful and complete answers to their questions. We compare three system architectures: the baseline retrieval-augmented generation system, the multi-index retrieval approach, and the better hybrid retrieval system. Performance is evaluated using a combination of retrieval metrics (Recall@K, Precision@K) and human-assessed answer quality scoring. The results presented in this chapter provide evidence for the impact of design choices such as semantic chunking, metadata usage, and hybrid retrieval on overall system performance.

### 5.1 Evaluation Dataset

A domain-specific evaluation dataset was created to test how well the proposed retrieval-augmented chatbot works. Because there isn't a public benchmark for answering questions on university websites, a custom set of representative user questions was made by hand based on the content of the university's official website.

There are  $N$  natural language questions in the dataset that are meant to show the real information needs of current and future students. The questions were written in a way that is meant to sound like how people usually talk to chat-based systems and cover a lot of different topics.

For each question, one or more ground-truth source URLs were found. These URLs point to the pages that have the information needed to answer the question correctly. They were

chosen from the pages that were scraped.

Table 5.1 presents a subset of the evaluation questions along with their assigned categories and corresponding ground-truth URLs. The complete list of questions is provided in Appendix A.

Table 5.1: Sample of evaluation questions and corresponding ground-truth URLs

| ID  | Question                                                                         | Ground-Truth URL(s)                         |
|-----|----------------------------------------------------------------------------------|---------------------------------------------|
| Q01 | Ποιος είναι ο διδάσκων του μαθήματος Λειτουργικά Συστήματα;                      | /studies/undergraduate/courses/ece318/      |
| Q03 | Ποια μαθηματα διδασκει η κυρία Τσαλαπάτα;                                        | /department/faculty/htsalapa/               |
| Q07 | Ποιο είναι το email της γραμματείας;                                             | /secretariat/                               |
| Q09 | Ποια προαπαιτούμενα έχει το μάθημα Ασφάλεια Δικτύων και Πληροφορικών Συστημάτων; | /studies/undergraduate/courses/ece468/      |
| Q27 | Πότε είναι οι διαλέξεις του μαθήματος Οργάνωση και Σχεδίαση Η/Υ;                 | /studies/undergraduate/fall-timetable/week/ |

The ground-truth URLs are the benchmark against which retrieval performance can be evaluated. A retrieved document chunk is considered relevant if it is sourced from one of the ground-truth URLs linked to the respective question and provides enough details to address the question at hand. Questions that did not have any relevant information in the scraped corpus were clearly marked as unanswerable and not included in the retrieval metric calculations.

## 5.2 Evaluation Metrics

We looked at the proposed chatbot system's performance in two ways: how well it retrieved information and how good the answers were. This difference makes it easy to tell the difference between mistakes made when retrieving information and mistakes made while generating answers.

### 5.2.1 Retrieval Metrics

We tested the system’s retrieval effectiveness by seeing how well it could find relevant document chunks when users asked for them. A retrieved chunk was considered relevant if it originated from one of the ground-truth URLs associated with the corresponding evaluation question and contained sufficient information to answer the question.

For each question, the top- $K$  retrieved chunks were analyzed, where  $K = 6$ . The following metrics were used.

#### Recall@K

Recall@K measures whether at least one relevant chunk appears among the top- $K$  retrieved results. It is defined as:

$$\text{Recall@K} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}(R_q^{(K)} \cap G_q \neq \emptyset) \quad (5.1)$$

where  $Q$  denotes the set of evaluation questions,  $R_q^{(K)}$  represents the set of top- $K$  chunks retrieved for question  $q$ ,  $G_q$  denotes the set of ground-truth relevant chunks for  $q$ , and  $\mathbb{I}(\cdot)$  is the indicator function.

Recall@K is particularly important in retrieval-augmented generation systems, as failure to retrieve relevant information prevents the language model from producing a correct answer regardless of its generative capabilities.

#### Precision@K

Precision@K measures the proportion of relevant chunks among the top- $K$  retrieved results and is defined as:

$$\text{Precision@K} = \frac{1}{|Q|} \sum_{q \in Q} \frac{|R_q^{(K)} \cap G_q|}{K} \quad (5.2)$$

Precision@K shows how much noise the system adds to the retrieval process. It is especially helpful for figuring out how re-ranking strategies and metadata-based filtering affect the results.

### 5.2.2 Answer Quality Metrics

Retrieval metrics show how well the system can find relevant information, but they don't show how good the responses are. So, the quality of the answers was judged by people using a scoring system. We looked at each answer in three ways: how relevant it was, how true it was, and how complete it was.

**Relevance** Relevance measures the degree to which the generated answer addresses the user's question. Answers that fully and directly addressed the information need received higher scores, while partially correct or off-topic answers received lower scores.

**Faithfulness** Faithfulness checks to see if the answer is based on the context that was found and doesn't add any information that isn't true or isn't based on anything. Answers that made claims that weren't backed up by the retrieved document chunks got lower scores.

**Completeness** Completeness checks to see if the answer has all the important information needed to fully answer the question. Answers that left out important information, like dates, conditions, or requirements, were punished in the appropriate way.

Each criterion was scored on a five-point scale, where a score of 1 indicates poor performance and a score of 5 indicates excellent performance. The final answer quality score for a response was computed as the arithmetic mean of the three individual criterion scores.

## 5.3 Evaluation Results

This section presents the quantitative evaluation results of the three system configurations considered in this study: a baseline retrieval-augmented generation system, a multi-index retrieval system, and a hybrid retrieval system. The systems were evaluated using retrieval metrics (Recall@6 and Precision@6) as well as human-evaluated answer quality metrics (relevance, faithfulness, and completeness), as described in Section 5.2.

### 5.3.1 Retrieval Performance

Table 5.3 summarizes the retrieval effectiveness of the evaluated systems in terms of average Recall@6 and Precision@6.



Table 5.2: Average retrieval and answer quality metrics across all evaluation questions

| System      | Recall@6    | Precision@6 | Relevance   | Faithfulness | Completeness |
|-------------|-------------|-------------|-------------|--------------|--------------|
| Baseline    | 0.70        | 0.25        | 3.86        | 3.00         | 2.55         |
| Multi-index | 0.37        | 0.17        | 3.80        | 2.33         | 2.17         |
| Hybrid      | <b>0.90</b> | <b>0.72</b> | <b>4.77</b> | <b>4.80</b>  | <b>3.93</b>  |

Table 5.3: Average retrieval performance across all evaluation questions

| System      | Recall@6    | Precision@6 |
|-------------|-------------|-------------|
| Baseline    | 0.70        | 0.25        |
| Multi-index | 0.37        | 0.17        |
| Hybrid      | <b>0.90</b> | <b>0.72</b> |

Both the baseline and multi-index configurations were considerably inferior to the hybrid retrieval system. The hybrid system was able to retrieve at least one relevant document chunk for 90% of the evaluation questions, with an average Recall@6 of 0.90. By comparison, the multi-index approach performed significantly worse with a Recall@6 of 0.37, whereas the baseline system achieved a Recall@6 of 0.70.

The pattern was similar for Precision@6. With a Precision@6 of 0.72, the hybrid system demonstrated that most of the retrieved chunks were pertinent. The multi-index system performed poorly with a Precision@6 of 0.17, while the baseline system showed significantly lower precision (0.25), indicating a high degree of retrieval noise.

These findings suggest that hybrid retrieval approach combined with semantic chunking significantly improve retrieval quality.

### 5.3.2 Answer Quality Evaluation

Answer quality was evaluated using a score from 1 to 5 along three dimensions: relevance, faithfulness, and completeness. Table 5.4 presents the average scores for each criterion on the five-point scale.

The hybrid system achieved the highest scores across all generation metrics. In particular, faithfulness scores were notably high (4.8), indicating that answers generated by the hybrid system were consistently grounded in the retrieved context. Completeness scores, while lower than relevance and faithfulness, still showed a notable improvement over the baseline and

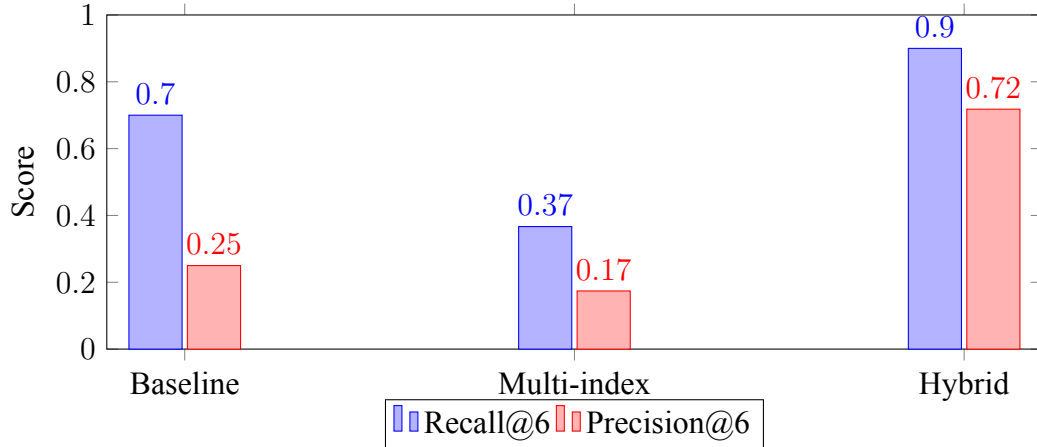


Figure 5.1: Comparison of retrieval performance across system configurations

Table 5.4: Average answer quality scores across all evaluation questions

| System      | Relevance   | Faithfulness | Completeness |
|-------------|-------------|--------------|--------------|
| Baseline    | 3.86        | 3.00         | 2.55         |
| Multi-index | 3.80        | 2.33         | 2.17         |
| Hybrid      | <b>4.77</b> | <b>4.80</b>  | <b>3.93</b>  |

multi-index approaches.

The multi-index system had the worst performance on both retrieval and answer quality metrics, which suggests that splitting the data into multiple independent indices makes semantic recall and overall system performance worse. These findings reinforce the importance of maintaining a unified retrieval space.

## 5.4 Error Analysis

The hybrid retrieval system had the best overall performance, but all of the system architectures that were tested had some failures. This section does a qualitative error analysis to find common mistakes that happen during the retrieval and answer generation stages. Comprehending these failure modes highlights the constraints of the suggested methodology and highlights opportunities for future improvements.

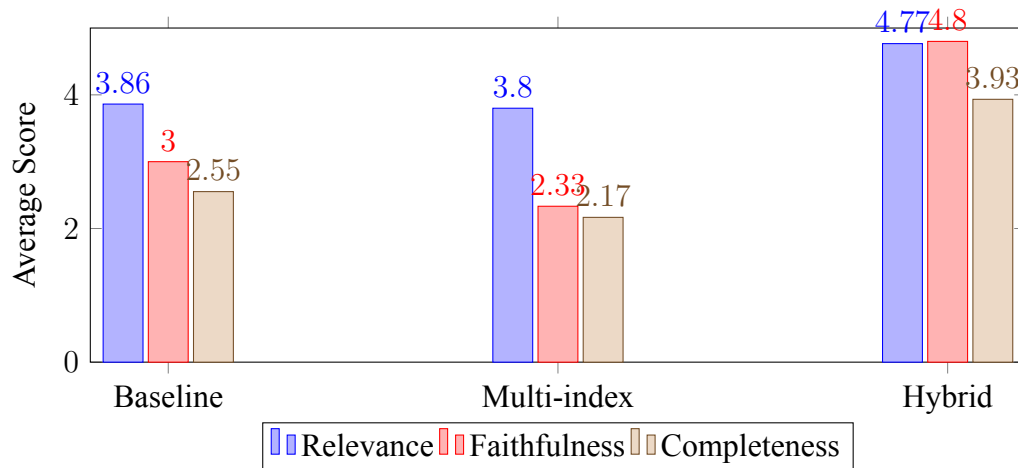


Figure 5.2: Comparison of answer quality metrics across system configurations

### 5.4.1 Retrieval Failures

Retrieval failures occurred when none of the retrieved document chunks contained the information required to answer the user question. Such failures directly resulted in low Recall@6 values and prevented the LLM from generating correct answers.

#### Multi-Index Fragmentation

A prominent source of retrieval failure was observed in the multi-index system configuration. By partitioning the document corpus into multiple independent indices based on page categories, the system introduced an implicit routing problem, requiring the system to select the appropriate index prior to retrieval. When incorrect routing decisions occur, relevant information may not be retrieved at all, resulting in zero relevant context for answer generation. User queries that spanned multiple semantic categories frequently resulted in retrieval from an incorrect index, leading to a complete absence of relevant chunks in the retrieved results. So, limiting retrieval to just one category-specific index makes it harder for the system to get this cross-domain information need.

The multi-index system has a Recall@6 score of only 0.37, which is much lower than the baseline’s score of 0.70 and the hybrid system’s score of 0.90. These results show that index partitioning can hurt semantic recall in areas where user queries don’t fit into predefined content categories.

## **Overly Generic Chunk Retrieval**

The retrieval of excessively generic document chunks in the baseline system frequently resulted in retrieval failures and incomplete responses. Fixed-size chunking without semantic boundaries produced chunks that lacked specific details like deadlines or procedural steps but contained general, introductory, and incomplete information. These sections showed a moderate degree of similarity to the query, but they were not enough to provide a complete response.

### **5.4.2 Answer Generation Errors**

Even in cases where relevant information was successfully retrieved, answer generation errors were observed. These errors were primarily related to the language model's handling of incomplete or noisy context.

#### **Incompleteness Due to Partial Context**

A common failure mode involved answers that were relevant and faithful but incomplete. This issue was particularly evident in the baseline and multi-index systems, which frequently retrieved only partial information required to fully answer a question. As a result, generated answers omitted important details such as exact dates, numerical values, or conditional requirements.

This phenomenon is reflected in the comparatively low completeness scores of 2.55 and 2.17 for the baseline and multi-index systems, respectively, compared to 3.93 for the hybrid system.

#### **Hallucinations Triggered by Missing Information**

Hallucinated content was primarily observed in cases where retrieval failed entirely or returned irrelevant context. In these cases, the LLM tried to guess what the right answer was based on what it already knew instead of what it had found. The multi-index system had a low faithfulness score of 2.33 because of this behavior.

The hybrid system, on the other hand, had a much higher faithfulness score of 4.8. This shows that better retrieval quality can reduce hallucination by connecting the language model to relevant source material.

### **5.4.3 Summary of Failure Modes**

Overall, the observed errors can be grouped into three primary categories: retrieval failures caused by corpus partitioning or generic chunking, answer incompleteness due to partial context, and hallucinations resulting from missing or irrelevant retrieved information.

The performance gains can be attributed to two primary design choices. First, semantic chunking was employed instead of fixed-size chunking. Chunks were constructed to represent coherent semantic units. Each chunk was enriched with extra context such as page titles and section headings, which significantly improved retrieval accuracy. Second, the system adopted a hybrid retrieval strategy combining dense vector search with keyword-based retrieval. The hybrid system was able to get both semantic similarity and exact lexical matches by combining keyword-based retrieval with re-ranking retrieved candidates. This made the responses more reliable and grounded.

Overall, the results show that the quality of the retrieval is the most important thing that affects how correct the answers are in retrieval-augmented generation systems. Semantic chunking combined with hybrid retrieval is a strong way to answer questions on complex websites, such as [e-ce.uth.gr](http://e-ce.uth.gr).



# Chapter 6

## User Interface and Front End Design

This chapter presents the user interface of the intended AI chatbot system, focusing on how the front end provides usability, accessibility and helps the user interact with the underlying RAG-based system.

### 6.1 User Interface Design Goals and Visual Identity

The chatbot user interface was designed as a **micro front-end component** that could be added to the official university website (e-ce.uth.gr). The main goal is to give users quick and easy access to related information without getting in the way of their navigation.

#### 6.1.1 Integration as a Micro Front-End

The chatbot is a chat widget that pops up and can be used from any page on the website. This design choice lets users ask for help at any time while using the site, without having to go to a separate help or frequently asked questions page.

The popup-based approach allows the chatbot to:

- stay out of sight when not in use
- keep out of the way of the user's journey on the website
- answer short questions that are related to the university

This design fits with the chatbot's job of providing information quickly, not replacing traditional ways of navigating a website.

### 6.1.2 Interaction Model

The interaction model follows a conversational chat paradigm similar to widely used AI chat systems such as ChatGPT and Gemini. This model was selected due to its familiarity to users, thereby reducing the learning curve and encouraging engagement. The interaction model is based on a conversational chat model, like many popular AI chat systems like ChatGPT and Gemini. This choice was made because users are already familiar with these systems, which makes it easier to learn and use our system.

Users enter their questions in a text box, and the system answers them one at a time in a chat-style format. This interaction method lets people ask questions in natural language, which is often vague or exploratory, especially for new students and visitors.

To minimize cognitive load, the interface was designed to:

- focus on a single interaction task at a time,
- avoid unnecessary visual elements,
- present responses in a clear and linear format.

### 6.1.3 Visual Design and Color Scheme

The chatbot's visual design is simple and modern, with a focus on readability and clarity over decoration. This method makes the chatbot more of a useful information tool than a complicated interactive system.

The color scheme comes from the university's official branding to make sure it looks the same as the main website. The main colors used in the interface are:

- **Blue (#347fb0)**, used for interactive elements and emphasis,
- **Light blue / gray (#d3e2eb)**, used for backgrounds and secondary interface elements,
- **White (#fafafa)**, used as the primary background color to maintain readability.

### 6.1.4 Clarity, Trust, and Transparency

To build trust with users and lower the chance of giving out false information, the system's responses are visually different from the user's input and include links to relevant university web pages. This design choice makes things more open and lets users check the infor-



mation the chatbot gives them on their own. The interface directly supports the underlying Retrieval-Augmented Generation (RAG) architecture by basing responses on official university sources.

## 6.2 Technology Stack

Here’s an overview of the technologies used in the UI of this project:

### 6.2.1 Core Framework & Language

| Technology      | Version | Purpose                                                                         |
|-----------------|---------|---------------------------------------------------------------------------------|
| React [13]      | 18.3.1  | The main UI library for building component-based interfaces                     |
| TypeScript [14] | 5.8.3   | Adds static typing to JavaScript for better developer experience and fewer bugs |
| Vite [15]       | 5.4.19  | Next-generation build tool with an extremely fast dev server and HMR            |

Table 6.1: Core framework and language

### 6.2.2 UI Component Library

| Technology                     | Purpose                                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------------------|
| shadcn/ui                      | Collection of reusable, accessible components built on Radix UI; configured via <code>components.json</code> |
| Radix UI                       | Unstyled, accessible UI primitives such as dialogs, tooltips, and popovers                                   |
| Lucide React                   | Icon library used for interface and category icons                                                           |
| class-variance-authority (CVA) | Utility for defining component variants (e.g., button sizes and styles)                                      |
| clsx + tailwind-merge          | Utilities for conditionally composing and merging Tailwind class names                                       |

Table 6.2: UI Component Libraries

### 6.2.3 State & Data Management

| Technology           | Purpose                                                                                                                        |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------|
| React Hooks          | <code>useState</code> , <code>useRef</code> , <code>useEffect</code> , and <code>useCallback</code> for local state management |
| TanStack React Query | Server state management (available but not heavily used in the chat implementation)                                            |
| Custom Hooks         | <code>useChatApi</code> encapsulates API logic separately from UI components                                                   |

Table 6.3: State and Data Management

6.2.4 Routing

| Technology       | Purpose                                                      |
|------------------|--------------------------------------------------------------|
| React Router DOM | Client-side routing and navigation between application pages |

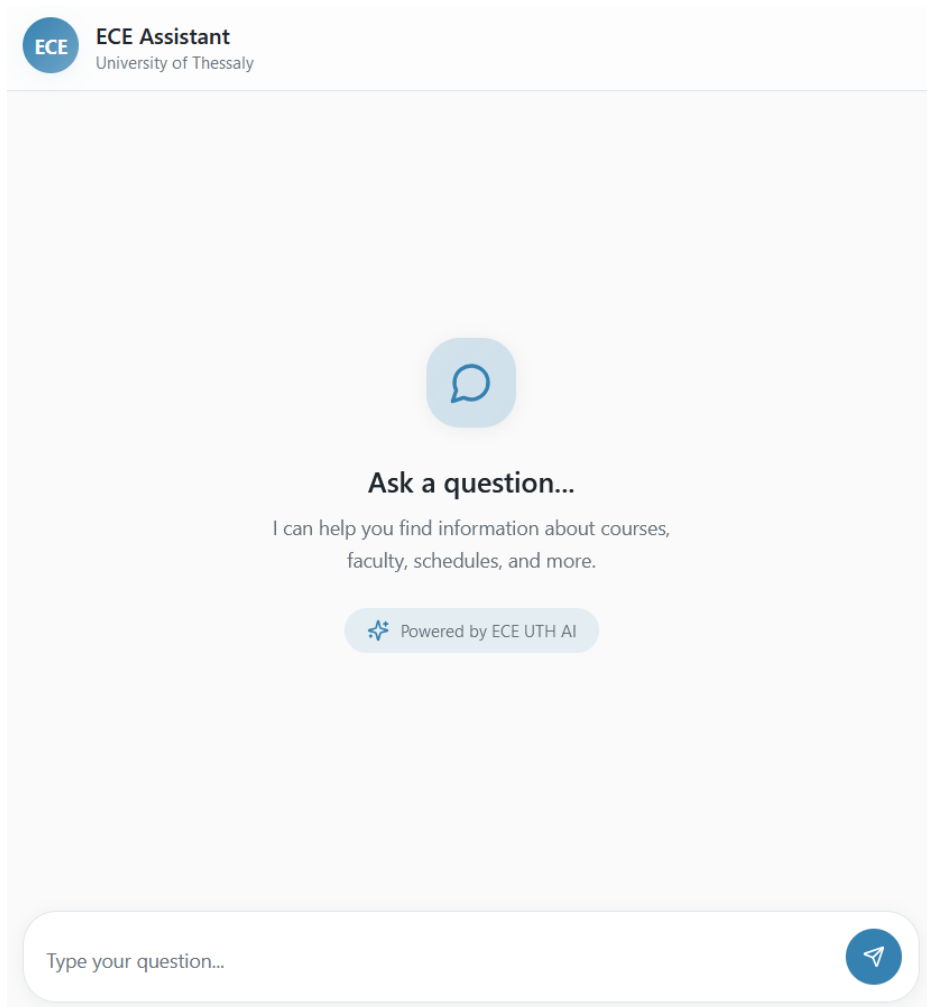
Table 6.4: Routing

6.2.5 Form Handling

| Technology          | Purpose                                       |
|---------------------|-----------------------------------------------|
| React Hook Form     | Form state management and validation handling |
| Zod                 | Schema-based validation and type inference    |
| @hookform/resolvers | Integrates Zod schemas with React Hook Form   |

Table 6.5: Form Handling Technologies

## 6.3 Core Interface Components



The image shows the initial (idle) state of the chatbot user interface, presented before any user interaction has taken place.

At the top-left corner, the chatbot identity is displayed, consisting of:

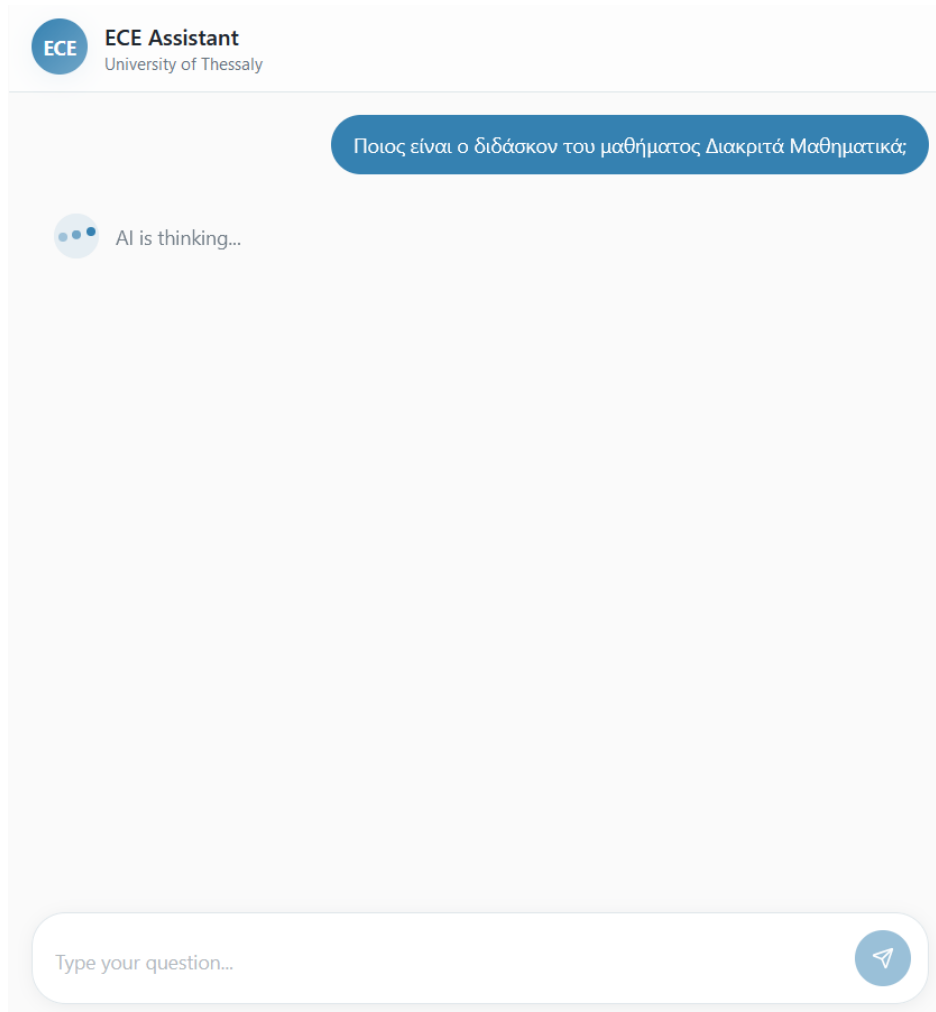
- A circular icon containing “ECE”
- The chatbot name “ECE Assistant”
- The subtitle “University of Thessaly”

The central area of the interface is intentionally minimal and uncluttered. It contains:

- A big speech bubble icon that makes the system look like a conversation
- A big prompt that says ”Ask a question...” to get people to take action
- A brief sentence that explains that the chatbot can help with things like courses, professors, schedules, and more

At the bottom of the screen, there is a fixed input area, consisting of:

- A text box with the words "Type your question..." in it
- A paper-plane icon shows a circular send button



The image depicts the chatbot interface during an active interaction, following the submission of a user query. In this example, the user asks a question in Greek regarding the instructor of a specific course ("Ποιος είναι ο διδάσκων του μαθήματος Διακριτά Μαθηματικά;")

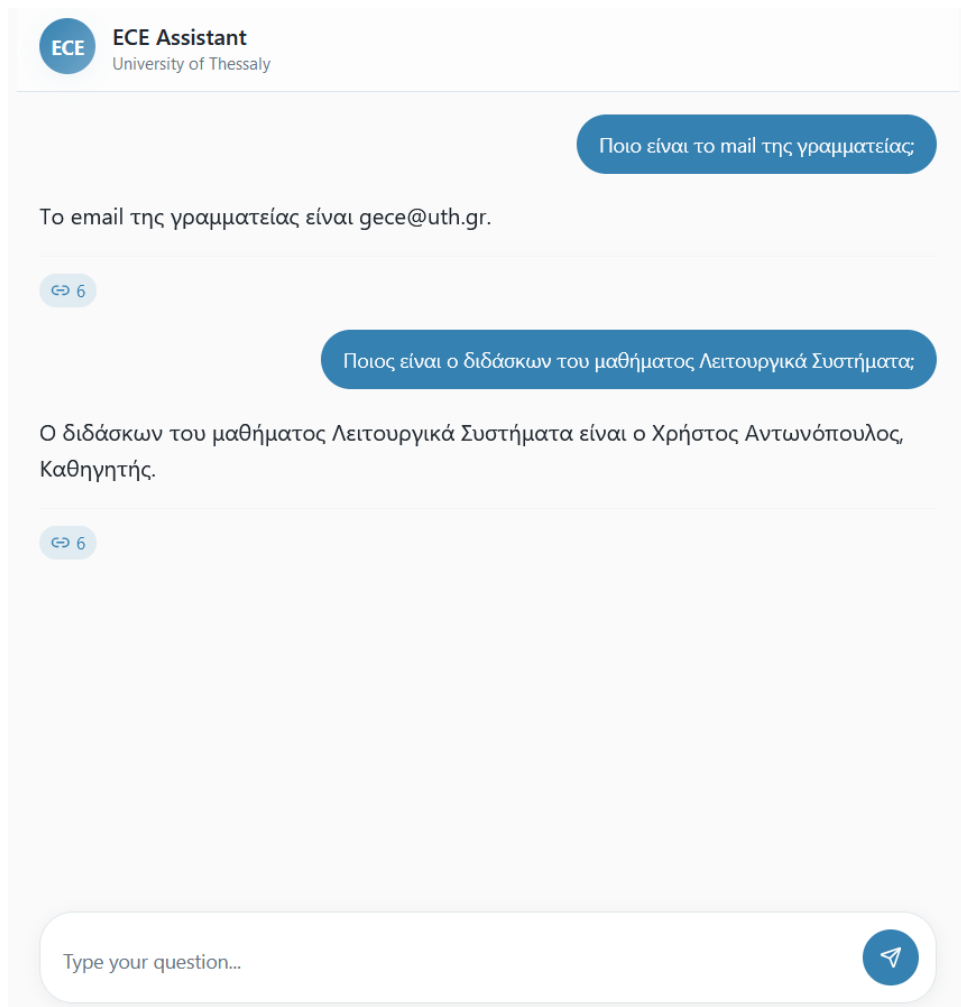
The user's question is shown in a message bubble that is aligned to the right, visually distinguished through the use of the primary blue color. This alignment and color difference make it easy to tell the difference between user input and system-generated responses. This is a common practice in conversational interfaces that makes them easier to read and flow.

As soon as the user submits a question, the interface shows a temporary system feedback indicator with the words "AI is thinking..." and a small animated icon. This part shows that the system has received the input and is working on the request.

Giving clear feedback during processing solves a major usability problem by:

- Preventing doubt about whether the query was sent in successfully
- Reducing perceived waiting time
- Increasing user confidence in system responsiveness

This feedback is especially important for Retrieval-Augmented Generation systems because the time it takes to get a response is longer because of the document retrieval and answer synthesis stages.



The image shows the chatbot interface after the system has generated and displayed responses to the user queries. This is the main functional part of the interaction, where the user sees the information that has been compiled in order to answer their question.

Each answer directly answers the user's question in a clear and punctual way. The interface keeps a record of the entire conversation, showing both question and answer pairs in the order. This design choice lets users:

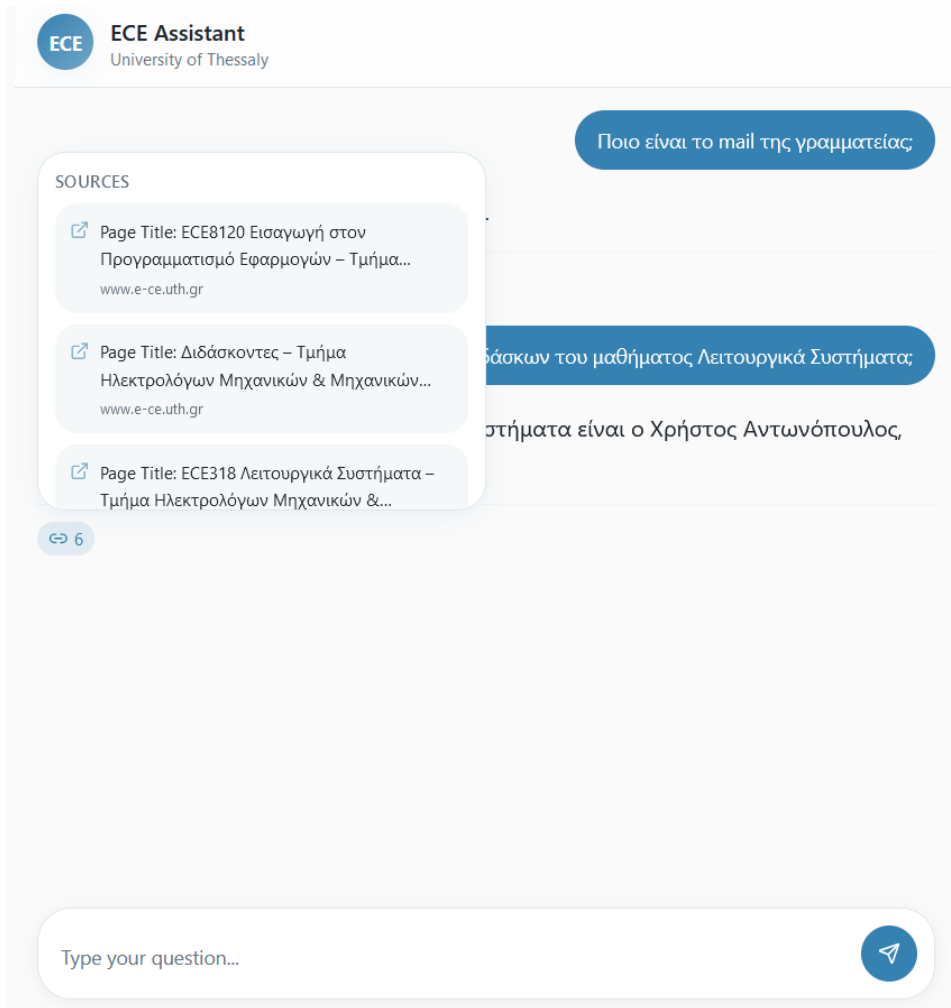
- Keep track of the context across several queries
- Refer back to previous responses
- Ask follow-up questions without losing your way

The layout of the interface stays the same during the whole interaction:

- The input field stays in the same place at the bottom of the screen
- Visual hierarchy and spacing are preserved
- There are no disruptive transitions or layout shifts

This stability reduces cognitive overhead and makes interactions more predictable, so users can focus on the content of the responses instead of how the interface works.

---



The picture shows the interface state where the user asks to see the sources that made up a system-generated response. When you hover on the source icon, a special overlay panel appears that shows the most relevant web pages that were used to generate the answer.

The source panel shows a list of six source entries, which are the top-ranked documents found during the retrieval phase of the Retrieval-Augmented Generation pipeline. Each entry has:

- The page title of the source document
- A direct link to the original university url
- An icon indicating that the link opens an external page

This design lets users check where the information came from without leaving the conversation unless they want to.

From a system point of view, the sources shown are the same as the context chunks that were found in response to the user's query. These chunks are chosen based on how similar



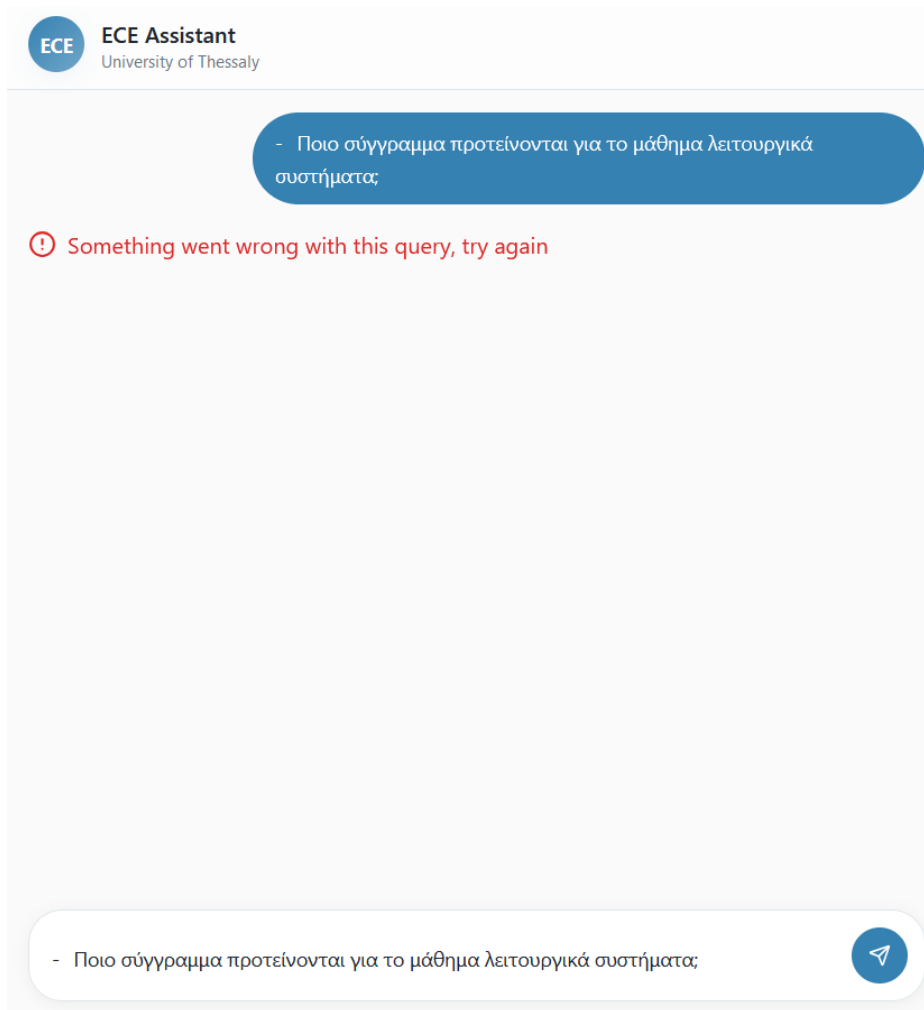
they are in meaning and how relevant they are to the question. Then, they are given to the large language model as context when it is generating an answer.

By exposing these sources at the interface level, the system creates a clear connection between:

- The retrieved content
- The generated response
- The authoritative university webpages

The explicit presentation of source links fixes a big problem with AI models: the lack of inherent explainability. By allowing users to verify responses against the e-ce.uth.gr website, the interface:

- Reduces the risk of misinformation
  - Increases user trust
  - Promotes the responsible use of content made by AI
-



The image presents the chatbot interface in an error state, occurring after an error has occurred. Such errors may arise from various factors, including temporary backend unavailability, retrieval failures, or timeouts during the generation process.

When something goes wrong, the interface shows a clear and short error message right in the conversation area. It says that the query couldn't be processed and asks the user to try again. The message is made more noticeable by using a red color and an alert icon, which makes it clear that the problem is with the system and not the user.

This approach ensures that errors are:

- Immediately visible
- Clearly distinguishable from normal system responses
- Communicated without technical terms

The user's question is put again on the input box, so they don't have to rewrite it in order to ask the system again,

# Chapter 7

## Technology Stack

This chapter presents the technology stack and implementation details of the proposed RAG-based chatbot system. It provides an overview of the software tools and frameworks used, describes the project structure and the responsibilities of each system component, and presents the integration of OpenAI models for embedding and response generation. These sections together describe the technology that the system is built on.

### 7.1 Overview of the Technology Stack

This project is built entirely in Python and uses a modular architecture to support data collection, semantic processing, retrieval, and answer generation within a Retrieval-Augmented Generation (RAG) framework. The chosen technologies focus on being flexible, easy to reproduce, and able to work with modern large language model (LLM) workflows.

The system uses web scraping tools to collect data, natural language processing libraries for text handling, vector similarity search for information retrieval, and LLMs for generating responses. A lightweight web backend exposes the chatbot functionality through an API.

Table 7.1 shows a list of the main technologies used in the implementation and what they do in the system.

| Layer                   | Technology                            | Purpose                                                                                      |
|-------------------------|---------------------------------------|----------------------------------------------------------------------------------------------|
| Programming Language    | Python [16]                           | Primary language for system implementation, orchestration, and integration of all components |
| Web Scraping            | requests [17], BeautifulSoup [18]     | Fetching and parsing HTML content from university websites                                   |
| Structured Data Parsing | xml.etree.ElementTree [19], json [20] | Handling XML sitemaps and structured data serialization                                      |
| Text Processing         | tiktoken [21]                         | Tokenization and token-length control for chunking and LLM context management                |
| Numerical Computing     | NumPy [22]                            | Efficient numerical operations for embedding storage and manipulation                        |
| Vector Search           | FAISS [23]                            | High-performance similarity search over dense embedding vectors                              |
| Keyword Retrieval       | rank_bm25 (BM25Okapi) [24]            | Lexical search to complement dense vector retrieval in a hybrid search setup                 |
| Embedding Model         | text-embedding-3-small [25]           | Generation of dense vector representations for semantic retrieval                            |
| Large Language Model    | GPT-4o-mini [26]                      | Answer generation using retrieved contextual information                                     |
| Backend Framework       | Flask [27]                            | API-based backend for handling user queries and responses                                    |
| Cross-Origin Support    | Flask-CORS [28]                       | Enabling cross-origin requests for frontend integration                                      |
| File System Handling    | pathlib [29]                          | Platform-independent file and directory management                                           |
| LLM API Client          | OpenAI Python SDK [30]                | Interaction with OpenAI models for embeddings and text generation                            |

Table 7.1: Technology stack used in the implementation of the RAG-based university chatbot

The chosen technology stack makes it possible to try out different ways of retrieving and chunking data while keeping the processing, retrieval logic, and answer generation separate. This modular design allows for expansion and makes evaluation easier, which is important for the experimental goals of this thesis.

## 7.2 Project Structure

There are three Python modules that form the system, and each one is responsible for a different part of the pipeline. The structure separates tasks that only need to be done once from those that need to be done during runtime, which makes things clearer and easier to maintain.

The `scrap_data.py` module is responsible for scraping the data from the university website performing semantic chunking. This script is executed once during the initial setup of the system to collect the needed information.

The `build_vector.py` module is responsible for making the embeddings and constructing the index vector based on the scraped data. This module is also intended to be executed once during the initial setup and rerun only when the website content is updated.

The `app.py` module implements the web backend of the system. It runs continuously at runtime and handles user queries by performing retrieval and answer generation. It exposes the chatbot functionality through an API interface.

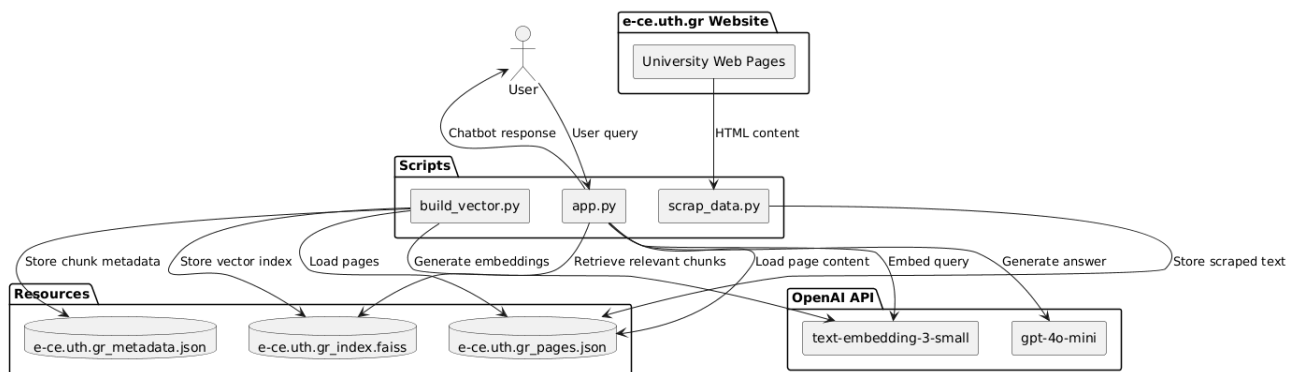


Figure 7.1: System Architecture

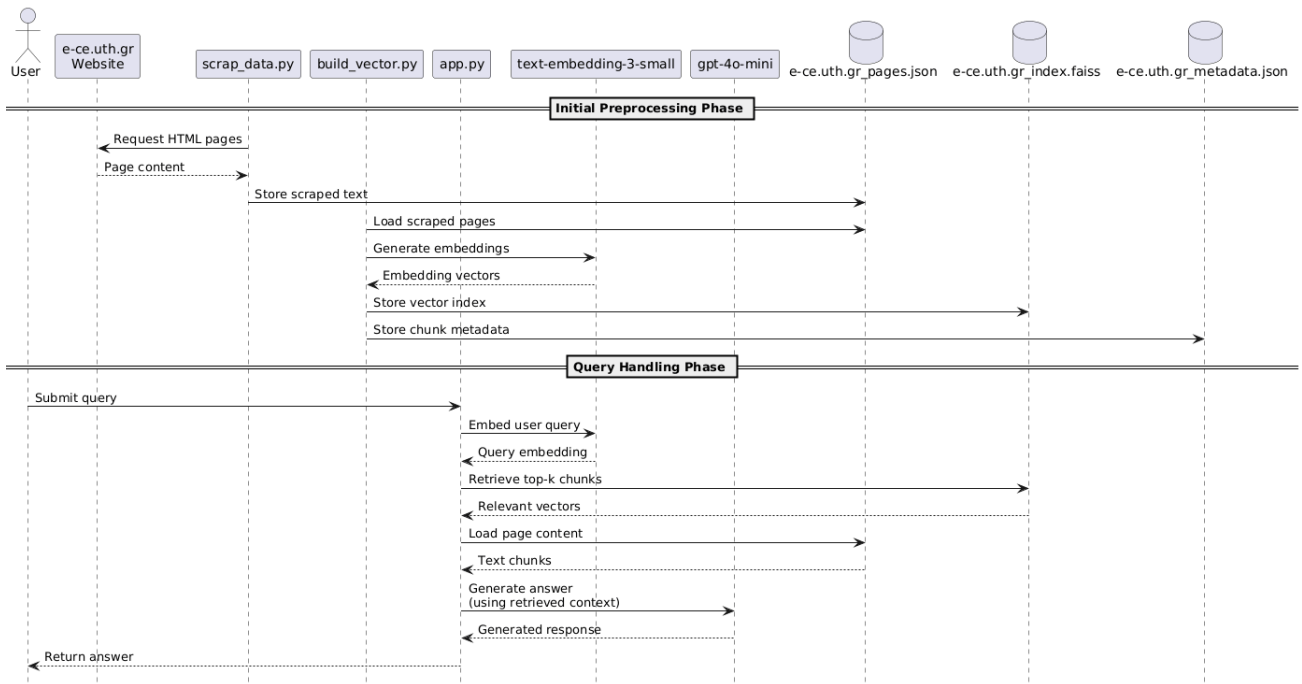


Figure 7.2: Sequence Diagram

In Figure 7.1 you can see the overall architecture of the proposed chatbot system, while in Figure 7.2 you can see the sequence diagram diagram of the interactions between the components of the system and the generated resources.

### 7.2.1 scrap\_data.py

The `scrap_data.py` module is responsible for the initial data collection and preprocessing stage of the system. It is executed once during system setup and produces a JSON file (`e-ce.uth.gr_pages.json`) containing semantically divided text chunks extracted from `e-ce.uth.gr`.

The module begins by retrieving all publicly accessible URLs from the website sitemap (<https://www.e-ce.uth.gr/sitemap.xml>). Sitemap indexes are handled recursively in order to discover nested sitemaps, while specific subsitemaps containing time-sensitive or irrelevant content (e.g., *announcements*) are explicitly excluded. This method makes sure that a lot of relevant informational pages are covered while cutting down on noise.

For each discovered URL, the corresponding HTML content is downloaded and parsed using an HTML parser. Then, the text is broken up into semantic sections based on structural HTML elements, such as headings (h1, h2, h3) and related text like paragraphs, list items, and tables. To keep structured information, tables are turned into text that can be read.

A token-based splitting strategy breaks each semantic section down into smaller pieces. To make sure that downstream embedding and language model constraints work, we put a limit of 400 tokens per chunk (a detailed explanation of tokenization and token counting is provided in Appendix B.4.). The language model's tokenizer is also used to count tokens, which lets you control the size of each chunk exactly.

For every resulting chunk, contextual metadata is embedded directly into the text, including the page title, section heading, and source URL. This contextual enrichment improves retrieval accuracy during later stages of the RAG pipeline. All chunks, along with their associated metadata, are stored in *e-ce.uth.gr\_pages.json*, which serves as the input for the vector index construction process.

Sample of a chunk:

```

1 {
2   "id": "https://www.e-ce.uth.gr/studies/undergraduate/courses/
   ece118/#sec4_chunk1",
3   "url": "https://www.e-ce.uth.gr/studies/undergraduate/courses/
   ece118/",
4   "page_title": "ECE118 ΔιακριτάΜαθηματικά-
   ΤμήμαΗλεκτρολόγωνΜηχανικών & ΜηχανικώνΥπολογιστών ",
5   "section": "Δομή ΠροαπαιτούμενωνΜαθημάτων ",
6   "text": "Page Title: ECE118 ΔιακριτάΜαθηματικά-
   ΤμήμαΗλεκτρολόγωνΜηχανικών & ΜηχανικώνΥπολογιστών \
   nSection: ΔομήΠροαπαιτούμενωνΜαθημάτων \nURL: https://www.e
   -ce.uth.gr/studies/undergraduate/courses/ece118/\n\nΕΘ()
   Εξάμηνο | Εξάμηνο 2 -ΕαρινόΕίδοςΜαθήματος |
   ΥποχρεωτικόΤύποςΜαθήματος | ΓενικούΥποβάθρουΣυν . Εβδ.
   ΩρώνΔιδασκαλίας | 4 ΩρεςΘεωρίας | 4 ΩρεςΕργαστηρίου | 0
   ΩρεςΦροντιστηρίου | 0 Μονάδες ECTS | 6 ΣελίδαΜαθήματος |
   https://eclass.uth.gr/courses/E-CE_U_204 ΥπεύθυνηΜαθήματος
   | ΔασκαλοπούλουΑσπασία , ΑναπληρώτριαΚαθηγήτρια E-mail:
   aspasia@uth.gr Διδάσκουσα | ΤσαλαπάταΧαρίκλεια , ΕΔΙΠ... E
   -mail: htsalapa@uth.gr Συγγράμματα | Βιβλίο [77106820]:
   Διακριτά"
7 }
```

Listing 7.1: Example of a chunk entry

### 7.2.2 build\_vector.py

The *build\_vector.py* module is responsible for transforming the semantically chunked text data into a vector index. After the data collection stage, this module makes the persistent resources required for efficient semantic retrieval.

The module loads the previously generated JSON file (*e-ce.uth.gr\_pages.json*), which contains the textual chunks extracted from the university website. Each chunk is embedded using the *text-embedding-3-small* model via the OpenAI API, producing dense vector representations suitable for semantic similarity search. To improve efficiency and comply with API constraints, embeddings are generated in batches.

The resulting embeddings are turned into a numerical array and normalized to unit length so that cosine similarity can be calculated using inner product. Then, a **FAISS index** (*IndexFlatIP*) is made and filled with the normalized vectors. This index is saved as *e-ce.uth.gr\_index.faiss* in a serialized format, which means it can be used again without having to be recomputed.

In addition to the vector index, a separate metadata file (*e-ce.uth.gr\_metadata.json*) is created to maintain the correspondence between vector positions and their source URLs. This separation of vector data and metadata simplifies index loading and retrieval during runtime.

The FAISS index and its associated metadata make up the output of this module. The web backend uses these two things to create the knowledge base that it uses to process user queries.

### 7.2.3 app.py

The *app.py* module implements the web backend of the chatbot system and is in charge of answering user questions at runtime. This part runs all the time, unlike the preprocessing modules and it combines retrieval, re-ranking, and answer generation into one API endpoint.

At startup, the module loads the precomputed text chunks from *e-ce.uth.gr\_pages.json*, the FAISS vector index from *e-ce.uth.gr\_index.faiss*, and the corresponding metadata from *e-ce.uth.gr\_metadata.json*. In addition, a **BM25** keyword index is constructed over the text corpus to support lexical retrieval.

When a user asks a question, the system uses a hybrid retrieval strategy to get the most information back. First, the query is embedded using the *text-embedding-3-small* model and used to perform a vector similarity search on the FAISS index. At the same time, the BM25 ranking function is used to do a keyword-based search. The results of both methods are



merged to form a candidate set of relevant text chunks.

To improve precision, the candidate chunks are subsequently re-ranked using a lightweight scoring procedure powered by the *gpt-4o-mini* model. Each candidate passage is scored based on its relevance to the user query, and only the highest-ranked passages are given as context for the answer generation.

The selected passages are then compiled into a prompt passed to the LLM with with explicit instructions to generate an answer exclusively based on the provided information. This prompt is designed to ensure faithfulness to the source content and reduce the risk of hallucination. The final response is returned in Greek, along with references to the source URLs from which the information was derived.

This runtime pipeline makes it possible to get accurate, efficient, and grounded answers while keeping the logic for retrieval and answer generation separate.

## 7.3 OpenAI - GPT

### 7.3.1 text-embedding-3-small

The `text-embedding-3-small` model is used to generate dense vector representations of text in the system that was suggested. These embeddings are the foundation of semantic retrieval, which lets the system find relevant passages based on meaning.

The embedding model's main job is to turn semantically meaningful text chunks into fixed-length numerical vectors. These vectors show how texts are related to each other in terms of meaning, so you can find passages that are similar in meaning even if they don't have the same words. The same model is also used to embed user queries while the program is running, to make sure that both documents and queries are in the same vector space.

There were a few reasons why the `text-embedding-3-small` model was chosen. The model achieves a good balance between semantic expressiveness and computational efficiency. This makes it a good choice for large sets of text chunks that are not too big. Second, it doesn't cost much to do the embedding operations over and over again when building the index and handling online queries, so there isn't much delay. Lastly, the model supports tokenization, which is necessary for chunk sizing and reproducible results.

There are a number of different embedding techniques that could have been applied to this system. Traditional word or sentence-level embedding models, such as *Word2Vec* or *GloVe*,

were not chosen because of their poor contextual awareness and inability to represent longer passages effectively. More recent transformer-based sentence embedding models, including *Sentence-BERT* and others provide better semantic representations, but they increase system complexity by requiring additional infrastructure.

Larger embedding models could be used; however, they are more expensive and more computationally complex without providing bigger benefits for the domain-specific, informational content of a university website. In contrast, *text-embedding-3-small* provides a trade-off between quality and efficiency. Its lower cost allows for scalable indexing and frequent query embedding, and its performance is ideal for medium-length text chunks that are typically found in our use case.

Furthermore, the model uses consistent tokenization, which makes preprocessing easier and guarantees reproducibility across pipeline stages, and integrates seamlessly with the language models used elsewhere in the system. These factors led to the selection of *text-embedding-3-small* as the best embedding model for the demands and limitations of this work.

### 7.3.2 gpt-4o-mini

The `gpt-4o-mini` model is used as the generative component of the proposed system, responsible for compiling natural language answers based on the given information. Within the Retrieval-Augmented Generation (RAG) framework, the role of the language model is not to provide knowledge independently, but to compose coherent and contextually grounded responses using the evidence supplied by the retrieval pipeline.

Several large language models were considered during the system design phase, including `gpt-4o` and `gpt-5o` that are more complex, thus more capable. While these models offer strong performance, their increased latency and cost make them less suitable for a real-time university chatbot, where responses must be delivered quickly and consistently. Moreover, because in the scope of the proposed system, the LLM isn't reasoning heavy, meaning that it doesn't need the computational power to search and reason, bigger models complexity would be an overkill for this generating heavy task.

The selection of `gpt-4o-mini` was decided by its favorable balance between response quality, efficiency, and operational cost. The model provides sufficient reasoning and language generation capabilities for the target domain. For such content, larger models do not

offer a significant qualitative advantage that would justify their additional overhead.

In addition, when configured with a low temperature parameter (in our system,  $temperature = 0$ ), `gpt-4o-mini` supports deterministic generation. Temperature controls the degree of randomness in the model’s token selection process. Higher temperature leads to more creativity and variability, while lower leads towards higher-probability tokens, producing more consistent and predictable responses. For a chatbot that provides information to users, to return consistent and accurate responses, it needs to be configured to a low temperature. This is aligned with the system’s goal of faithfulness and dependability as opposed to creative language generation.

By using `gpt-4o-mini` with a prompt that is grounded on the given context, the system reduces hallucination and produces an answer based on actual facts of the official content of the university website. This design choice allows the LLM to function as a controlled synthesis component rather than an open-ended knowledge source.

### 7.3.3 Cost

This subsection presents an empirical cost analysis of the OpenAI models used in the proposed system. Costs are separated into offline preprocessing costs and online query-handling costs. All estimates are based on token (appendix:B.4) usage measurements collected during system operation and on model pricing as of January 2026[31]. Pricing policies may change over time, so we are going to provide both order-of-magnitude estimates and exact deployment costs.

#### Offline Embedding Cost

During the preprocessing stage, all semantically chunked documents are embedded using the `text-embedding-3-small` model. Let  $N$  denote the total number of chunks and  $T_c$  the maximum number of tokens per chunk. In the current dataset:

$$N = 29,232, \quad T_c = 400$$

The total number of embedding tokens is therefore:

$$T_{\text{embed}} = N \times T_c = 29,232 \times 400 = 11,692,800 \text{ tokens}$$

Given a price of \$0.02 per million tokens, the total offline embedding cost is computed as:

$$\text{Cost}_{\text{embed}} = \frac{T_{\text{embed}}}{10^6} \times 0.02 = 11.6928 \times 0.02 = 0.233856 \text{ USD}$$

This cost is incurred once during index construction and does not affect runtime query handling.

### Online Query Cost

Each user query triggers multiple OpenAI API calls during runtime. Specifically, the system performs:

- 1 query embedding using `text-embedding-3-small`
- 20 reranking calls using `gpt-4o-mini`
- 1 final answer generation call using `gpt-4o-mini`

Token usage was empirically measured over a sample of ten representative user queries. Table 7.2 summarizes the average token consumption per query.

| Stage                 | Input Tokens  | Output Tokens |
|-----------------------|---------------|---------------|
| Query embedding       | 45            | –             |
| Reranking (all calls) | 18,947        | 58            |
| Answer generation     | 1,723         | 100           |
| <b>Total</b>          | <b>20,715</b> | <b>158</b>    |

Table 7.2: Average token usage per user query

### Cost per Query Estimation

Let  $P_{\text{in}}$  and  $P_{\text{out}}$  denote the prices per million input and output tokens for `gpt-4o-mini`, respectively. As of January 2026:

$$P_{\text{in}} = 0.15 \text{ USD}, \quad P_{\text{out}} = 0.60 \text{ USD}$$

The cost of reranking per query is therefore:

$$\text{Cost}_{\text{rerank}} = \frac{18,947}{10^6} \times P_{\text{in}} + \frac{58}{10^6} \times P_{\text{out}}$$

$$\text{Cost}_{\text{rerank}} = \frac{18,947}{10^6} \times 0.15 + \frac{58}{10^6} \times 0.60$$

$$\text{Cost}_{\text{rerank}} = 0.018947 + 0.0000348$$

$$\text{Cost}_{\text{rerank}} = 0.0189818 \text{USD}$$

Similarly, the cost of answer generation per query is:

$$\text{Cost}_{\text{answer}} = \frac{1,723}{10^6} \times P_{\text{in}} + \frac{100}{10^6} \times P_{\text{out}}$$

$$\text{Cost}_{\text{answer}} = \frac{1,723}{10^6} \times 0.15 + \frac{100}{10^6} \times 0.60$$

$$\text{Cost}_{\text{answer}} = 0.00025845 + 0.00006$$

$$\text{Cost}_{\text{answer}} = 0.00031845 \text{USD}$$

The query embedding cost is negligible in comparison and is given by:

$$\text{Cost}_{\text{query\_embed}} = \frac{45}{10^6} \times 0.02 = 0.0000009 \text{USD}$$

The total cost per query is:

$$\text{Cost} = \text{Cost}_{\text{rerank}} + \text{Cost}_{\text{answer}} + \text{Cost}_{\text{query\_embed}}$$

$$\text{Cost} = 0.0189818 + 0.00031845 + 0.0000009 = 0.01930115 \text{USD}$$

**Discussion**

The results show that the reranking stage is the most expensive part of the process, using up more than 90% of the total tokens per query. This behavior is to be expected because reranking requires many calls to the language model that are heavy on prompts and process a lot of contextual information while producing very little output.

On the other hand, generating the final answer and embedding the query don't add much to the total cost. These results show that there is a basic trade-off between the accuracy of retrieval and the cost of doing business. Reranking makes answers much more relevant and trustworthy, but it is also the main cost driver of the system. So, future improvements might look into cutting down on the reranking costs by making it into less calls to GPT and/or switching out heavy models for lighter ones.

# Chapter 8

## Conclusion

### 8.1 Synopsis and Final Conclusions

This thesis presented the design, implementation, and evaluation of a retrieval-augmented generation (RAG) chatbot for a university information system. The primary objective was to investigate whether a carefully designed retrieval pipeline could improve the accuracy, faithfulness, and completeness of LLM responses when answering real-world student queries.

A complete end-to-end system was developed, incorporating automated website scraping, semantic chunking of HTML content, embedding-based indexing with metadata enrichment, query processing, retrieval, and grounded answer generation. A web-based frontend was additionally implemented to show how easy it is to use the system as an interactive interface for getting to institutional information.

Three system configurations were implemented and compared: a baseline vector-based retrieval system, a multi-index one, and a hybrid retrieval system combining semantic chunking, metadata-aware retrieval, and hybrid search techniques. An evaluation dataset consisting of student questions for the system was constructed and system performance was evaluated using retrieval effectiveness metrics and human-evaluated answer quality metrics.

The evaluations showed that the hybrid retrieval system did better than both the baseline and multi-index architectures on all of the evaluation metrics. The hybrid approach achieved higher recall and precision scores and its answers were deemed as more relevant, faithful and complete. The multi-index architecture performed worse, demonstrating that splitting the data into multiple independent indices introduces retrieval failures in complex user queries.

In general, the results of this thesis show that retrieval design is a key factor in how

well RAG systems work. Semantic chunking, unified indexing and hybrid retrieval strategies were shown to reduce hallucinations and improve answer quality. This work demonstrates the feasibility of deploying RAG-based chatbots as reliable and practical interfaces for complex university information systems and provides empirical guidance for future implementations in similar domains.

## 8.2 Future Work

The proposed system works very well, but there are a number of ways for future work that could make it even more stable, efficient, and useful in real-world situations.

One promising extension is to use automatic query rewriting to preprocess queries. Using an LLM to change user queries into clearer and more informative forms could make retrieval even better, especially for short or unclear questions that students often ask each other.

Another important direction concerns retrieval re-ranking efficiency. Although language model-based re-ranking improves precision, it incurs significant computational cost. Future work may explore methods to decrease the number of necessary LLM calls, including selective re-ranking, cascaded ranking techniques, or the employment of more advanced models to execute ranking and generation with fewer inference steps.

For real-world use, it is essential to implement a mechanism that periodically detect changes and additions within the university website, in order to scrap them and embed them onto the index. That way the chatbot would be up to date with the latest information of the university.

Another useful improvement is making the system work with multiple languages. In particular, allowing the chatbot to handle English-language questions and answers would make it more available to users from other countries and allow it to be used in a wider range of academic settings.

Additionally, keeping conversational history and using it not only to enhance user experience by enabling more natural language but also using into the retrieval and generation process so that it can improve the system's performance.

Finally, future research might look into more advanced ways to retrieve information, such as adaptive retrieval strategies, learned sparse–dense retrieval combinations and cross-document reasoning methods, making the retrieval accuracy even better in fields where useful



---

information is spread out over many documents.



# Bibliography

- [1] Mutammimul Ula, Richki Hardi, and Irwandi Hipiny. An improved structure for academic information services through ai chatbots. *Journal of Engineering Science & Technology Review*, 16(5), 2023.
- [2] Shanshan Yang and Chris Evans. Opportunities and challenges in using ai chatbots in higher education. In *Proceedings of the 2019 3rd International Conference on Education and E-Learning*, pages 79–83, 2019.
- [3] Truong Dang-Huy Le Hoanh Su, Tran Thi-Yen-Linh, Nguyen Thi-Duyen-Ngoc, Ly Bao-Tuyen, and Nguyen Ha-Phuong-Truc. Development of an ai chatbot to support admissions and career guidance for universities. *International Journal of Emerging Multidisciplinary Research*, 4(2):11–17, 2020.
- [4] R Thamilselvan, P Natesan, E Gothai, SE Naveenkumar, JK Shanthosh, and S Vigneshwaran. Developing an ai-driven chatbot for enhanced college website support using machine learning. In *2024 International Conference on Expert Clouds and Applications (ICOECA)*, pages 719–726. IEEE, 2024.
- [5] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian. A comprehensive overview of large language models. *ACM Trans. Intell. Syst. Technol.*, 16(5), August 2025.
- [6] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. A survey on evaluation of large language models. *ACM Trans. Intell. Syst. Technol.*, 15(3), March 2024.

- [7] Kimberly Hau, Safwat Hassan, and Shurui Zhou. Llms in mobile apps: Practices, challenges, and opportunities. In *2025 IEEE/ACM 12th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*, pages 3–14, 2025.
- [8] Irene Weber. Large language models as software components: A taxonomy for llm-integrated applications. *arXiv preprint arXiv:2406.10300*, 2024.
- [9] Kailash A. Hambarde and Hugo Proença. Information retrieval: Recent advances and beyond. *IEEE Access*, 11:76581–76604, 2023.
- [10] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore, December 2023. Association for Computational Linguistics.
- [11] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. Retrieval-augmented generation for ai-generated content: A survey. *Data Science and Engineering*, January 2026.
- [12] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- [13] Meta Platforms, Inc. React: A javascript library for building user interfaces. <https://react.dev>, 2024. Accessed: 2026-02-04.
- [14] Gavin Bierman, Martín Abadi, and Mads Torgersen. Understanding typescript. In *European Conference on Object-Oriented Programming*, pages 257–281. Springer, 2014.
- [15] Evan You and Vite Contributors. Vite: Next generation frontend tooling. <https://vitejs.dev>, 2024. Accessed: 2026-02-04.
- [16] Python Software Foundation. Python programming language. <https://www.python.org>, 2024. Accessed: 2026-02-04.
- [17] Kenneth Reitz. Requests: Http for humans. <https://docs.python-requests.org>, 2024. Accessed: 2026-02-04.

- [18] Leonard Richardson. Beautiful soup documentation. *April*, 2007.
- [19] Python Software Foundation. xml.etree.elementtree — the elementtree xml api. <https://docs.python.org/3/library/xml.etree.elementtree.html>, 2024. Accessed: 2026-02-04.
- [20] Python Software Foundation. json — json encoder and decoder. <https://docs.python.org/3/library/json.html>, 2024. Accessed: 2026-02-04.
- [21] OpenAI. tiktoken: Fast bpe tokeniser for openai models. <https://github.com/openai/tiktoken>, 2024. Accessed: 2026-02-04.
- [22] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.
- [23] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. 2024.
- [24] Dorian Brown. rank\_bm25: A collection of bm25 algorithms. [https://github.com/dorianbrown/rank\\_bm25](https://github.com/dorianbrown/rank_bm25), 2023. Accessed: 2026-02-04.
- [25] OpenAI. Openai embedding models. <https://platform.openai.com/docs/guides/embeddings>, 2024. Accessed: 2026-02-04.
- [26] OpenAI. Gpt-4o-mini model documentation. <https://platform.openai.com/docs/models>, 2024. Accessed: 2026-02-04.
- [27] Pallets Project. Flask web framework. <https://flask.palletsprojects.com>, 2024. Accessed: 2026-02-04.
- [28] Cory Dolphin. Flask-cors. <https://flask-cors.readthedocs.io>, 2024. Accessed: 2026-02-04.

- [29] Python Software Foundation. pathlib — object-oriented filesystem paths. <https://docs.python.org/3/library/pathlib.html>, 2024. Accessed: 2026-02-04.
- [30] OpenAI. Openai python sdk. <https://github.com/openai/openai-python>, 2024. Accessed: 2026-02-04.
- [31] OpenAI. Pricing — openai platform. <https://platform.openai.com/docs/pricing>, 2024. Accessed: 2026-02-04.

# **APPENDICES**





# Appendix A

## Evaluation Data Set

This appendix contains all of the questions used to test how well the proposed chatbot system can retrieve information and generate answers. Each question has a main category and one or more ground-truth URLs that point to the pages that have the information needed to answer the question.

The questions were made by hand to reflect the real information needs of current and future students. They were then linked to ground-truth sources chosen from the scraped university website corpus. As explained in Section 5.1, these ground-truth URLs are the standard for judging how well retrieval works.

Table A.1: Complete evaluation question set with ground-truth URLs

| ID  | Question                                                    | Ground-Truth URL(s)                                |
|-----|-------------------------------------------------------------|----------------------------------------------------|
| Q01 | Ποιος είναι ο διδάσκων του μαθήματος Λειτουργικά Συστήματα; | <i>/studies/undergraduate/courses/ece3</i>         |
| Q02 | Πού γίνονται τα εργαστήρια του Προγραμματισμού I;           | <i>/studies/undergraduate/courses/ece1</i>         |
| Q03 | Ποια μαθήματα διδάσκει η κυρία Τσαλαπάτα;                   | <i>/department/faculty/htsalapa/</i>               |
| Q04 | Τι μαθήματα γίνονται τη Δευτέρα στις 9;                     | <i>/studies/undergraduate/fall-timetable/week/</i> |
| Q05 | Ποιο είναι το email της κυρίας Τσαλαπάτας;                  | <i>/department/faculty/htsalapa/</i>               |
| Q06 | Ποιο είναι το email της γραμματείας;                        | <i>/secretariat/</i>                               |
| Q07 | Ποιες είναι οι ώρες λειτουργίας της γραμματείας;            | <i>/secretariat/</i>                               |

| ID  | Question                                                                               | Ground-Truth<br>URL(s)                                                                                                  |
|-----|----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Q08 | Ποιο σύγγραμμα προτείνεται για το μάθημα Λειτουργικά Συστήματα;                        | <a href="/studies/undergraduate/courses/ece3">/studies/undergraduate/courses/ece3</a>                                   |
| Q09 | Ποια προαπαιτούμενα έχει το μάθημα Ασφάλεια Δι-<br>κτύων και Πληροφοριακών Συστημάτων; | <a href="/studies/undergraduate/courses/ece4">/studies/undergraduate/courses/ece4</a>                                   |
| Q10 | Ποια είναι τα μαθησιακά αποτελέσματα του μαθήματος<br>Τεχνικό Σχέδιο;                  | <a href="/studies/undergraduate/courses/ece1">/studies/undergraduate/courses/ece1</a>                                   |
| Q11 | Πότε ξεκινάει το εαρινό εξάμηνο;                                                       | <a href="/studies/academic-calendar/">/studies/academic-<br/>calendar/</a>                                              |
| Q12 | Πού μπορώ να βρω το πρόγραμμα της εξεταστικής;                                         | <a href="/category/exams-schedule/undergraduate-studies/">/category/exams-<br/>schedule/undergraduate-<br/>studies/</a> |
| Q13 | Πού βρίσκεται το τμήμα HMMY;                                                           | <a href="/contact-info/">/contact-info/</a>                                                                             |
| Q14 | Ποια είναι τα μαθήματα του Γ' εξαμήνου;                                                | <a href="/studies/undergraduate/">/studies/undergraduate/</a>                                                           |
| Q15 | Ποιο είναι το πρόγραμμα σπουδών;                                                       | <a href="/studies/undergraduate/">/studies/undergraduate/</a>                                                           |
| Q16 | Αν έχω βαθμό διπλώματος 5.1, ποιος είναι ο χαρακτηρι-<br>σμός της επίδοσής μου;        | <a href="/studies/undergraduate/">/studies/undergraduate/</a>                                                           |
| Q17 | Τι γίνεται σε περίπτωση αντιγραφής;                                                    | <a href="/studies/undergraduate/">/studies/undergraduate/</a>                                                           |
| Q18 | Τι πρέπει να κάνω για την εξειδίκευση στην επιστήμη<br>δεδομένων;                      | <a href="/studies/certificates/data-science-certificate/">/studies/certificates/data-<br/>science-certificate/</a>      |
| Q19 | Πόσα ECTS χρειάζονται για να πάρω πτυχίο;                                              | <a href="/studies/undergraduate/">/studies/undergraduate/</a>                                                           |
| Q20 | Πώς γίνεται η αξιολόγηση των φοιτητών στο μάθημα<br>Αλγόριθμοι CAD;                    | <a href="/studies/undergraduate/courses/ece4">/studies/undergraduate/courses/ece4</a>                                   |
| Q21 | Μπορώ να μην παρακολουθήσω τα Αγγλικά εάν έχω πτυ-<br>χίο B2;                          | <a href="/studies/undergraduate/courses/ece1">/studies/undergraduate/courses/ece1</a>                                   |
| Q22 | Ποιος είναι ο πρόεδρος του τμήματος;                                                   | <a href="/department/administration/">/department/administration/</a>                                                   |
| Q23 | Ποια είναι τα μεταπτυχιακά προγράμματα;                                                | <a href="/studies/postgraduate/">/studies/postgraduate/</a>                                                             |
| Q24 | Τι είναι τα ευφυή δίκτυα ηλεκτρικής ενέργειας;                                         | <a href="/studies/postgraduate/smart-grid-energy-systems/">/studies/postgraduate/smart-<br/>grid-energy-systems/</a>    |
| Q25 | Ποιες είναι οι προϋποθέσεις για συμμετοχή στο Erasmus;                                 | <a href="/studies/mobility/">/studies/mobility/</a>                                                                     |

| ID  | Question                                                         | Ground-Truth<br>URL(s)                             |
|-----|------------------------------------------------------------------|----------------------------------------------------|
| Q26 | Πότε είναι οι ορκωμοσίες της άνοιξης;                            | <i>/studies/academic-calendar/</i>                 |
| Q27 | Πότε είναι οι διαλέξεις του μαθήματος Οργάνωση και Σχεδίαση Η/Υ; | <i>/studies/undergraduate/fall-timetable/week/</i> |
| Q28 | Τι είναι το ειδικό θέμα;                                         | <i>/studies/undergraduate/courses/ece5</i>         |
| Q29 | Πότε μπορώ να δηλώσω διπλωματική εργασία;                        | <i>/studies/undergraduate/</i>                     |
| Q30 | Ποιες φοιτητικές ομάδες υπάρχουν;                                | <i>/student-teams/</i>                             |



# Appendix B

## GPT Tokens

This appendix gives a short overview of what tokens are and how they work in Large Language Model (LLM) systems. Tokenization is a basic process that has a direct effect on the inputs, outputs, and costs of running a model.

### B.1 Definition of Tokens

A token is the smallest piece of text that an LLM can process. Tokens do not correspond 1:1 to words or characters. Instead, they represent subword units generated by a tokenizer that breaks text into fragments based on patterns learned during model training.

For example, common or short words may correspond to a single token, while longer or more complex words may be split into multiple tokens. Punctuation symbols, numbers, and special characters are also represented as individual tokens.

### B.2 Tokenization Process

Before text is processed by an LLM, it is converted into a sequence of tokens using a tokenizer module. In this thesis, token counts are computed using the same tokenizer used by the OpenAI models (*tiktoken*), ensuring consistency between all stages of the system.

The tokenization process can be formally described as a mapping:

$$f : \text{Text} \rightarrow \{t_1, t_2, \dots, t_n\}$$

where each  $t_i$  represents an individual token. The total number of tokens  $n$  determines the length of the model input.

### B.3 Tokens and Model Constraints

Large Language Models have strict limits on the number of tokens they can use, which limits how long prompts and outputs can be. These limits change how the system is designed, including the size of chunks, the depth of retrieval, and the way prompts are written. In retrieval-augmented systems, it is important to control the number of tokens so that the system is both complete and efficient.

### B.4 Tokens and Cost

The cost of using language models and embedding models is directly proportional to the number of tokens processed. For embedding models, cost is typically based on the total number of input tokens. For generative models, pricing distinguishes between input (prompt) tokens and output (generated) tokens.

Let  $T_{\text{in}}$  denote the number of input tokens and  $T_{\text{out}}$  the number of output tokens. The total cost is then computed as:

$$\text{Cost} = \frac{T_{\text{in}}}{10^6} \cdot P_{\text{in}} + \frac{T_{\text{out}}}{10^6} \cdot P_{\text{out}}$$

where  $P_{\text{in}}$  and  $P_{\text{out}}$  represent the prices per million input and output tokens, respectively.